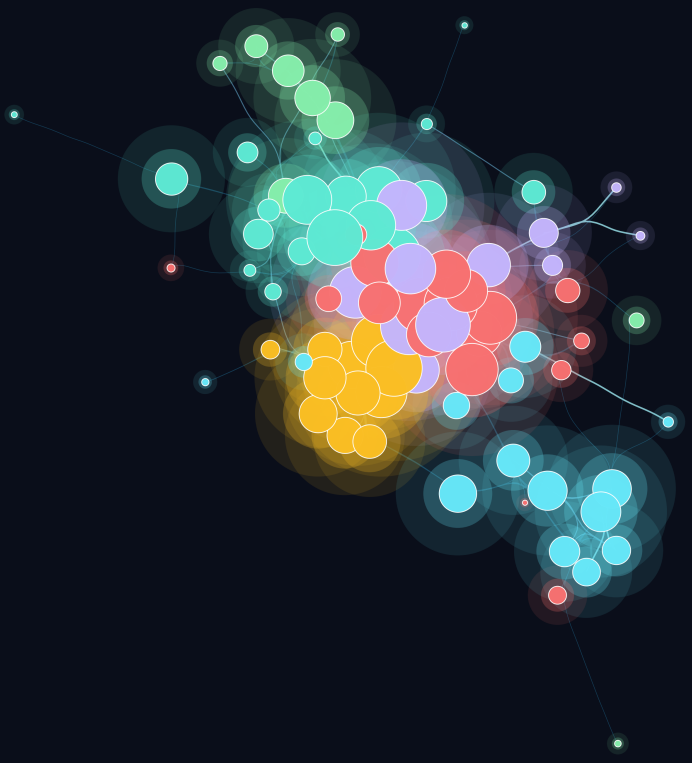
Accessibility Benchmark

Aquavena for the Visually Impaired

by Adrien SALES



github.com/adriens/aquavena linkedin.com/in/adrien-sales

May 31, 2026

Accessibility Benchmark
Aquavena for the Visually Impaired

Comparing `adriens.github.io/aquavena` vs `aquavena.nc`
Web Accessibility · Low Vision · WCAG 2.1 AA · New Caledonia

Adrien SALES

x.com/rastadidi linkedin.com/in/adrien-sales
github.com/adriens/aquavena

May 31, 2026

Abstract

*This report automatically compares the web accessibility of **adriens.github.io/aquavena** — designed for visually impaired users — against the source website **aquavena.nc**, a dietetic meal delivery service in New Caledonia. Standard CLI tools (pa11y-ci, Lighthouse, axe-core, webhint) are used as a common reference. Results show a **100/100 Lighthouse score and 0 WCAG 2.1 AA errors** for our site, versus 91/100 and 67 errors for the official website. All audit data is persisted in a DuckDB database for longitudinal tracking.*

Keywords: civitech, tech for good, inclusive design, digital inclusion, accessibility, WCAG, visually impaired, social impact, assistive technology, elderly care, aging population, senior accessibility, human-centered design, design thinking, New Caledonia.

Table of contents

1 ❤ Inception

This project was not born in a research lab or a corporate accessibility audit. It started at a kitchen table, from a very simple and concrete situation.

My mother has been a loyal Aquavena customer for years. Every week she orders her meals from their service, and she genuinely enjoys the food. But she is visually impaired — and as her condition evolved, consulting the weekly menus on the official website became increasingly difficult, then impossible to do on her own. The layout, the font sizes, the colour contrasts, the absence of any audio alternative: none of it was designed with her in mind.

The consequence was small but recurring, and that is precisely what made it significant. Every week, she had to wait for someone to be available — a family member, a neighbour — to read the menus out loud to her. She lost the freedom to plan her own meals at her own pace. A minor dependency, perhaps, but one that repeated itself week after week, quietly eroding her autonomy.

I am a software engineer. I realised that a few hours of work could give that autonomy back to her: a dedicated page she could open on her tablet, with large text, high contrast, a dark mode for tired eyes, and a button to have the menus read aloud in her own language. Something she could use alone, at any hour, without asking anyone for help.

That is why this site exists. It is not a product, not a startup, not a statement. It is a small act of care, built with the tools I know, for the person who matters most. The accessibility work documented in this report is a direct consequence of that intent: if the site is meant to help someone with low vision, it had better actually be accessible.

A note to the Aquavena team, should they ever read this: there is no competition here, no monetisation, no hidden agenda. Every menu on this site links back to aquavena.nc to place orders. This is simply a more accessible front door to a service my mother loves.

2 📈 Executive Dashboard — Delivery KPIs

This dashboard reads directly from `benchmark.duckdb` at render time and produces the KPIs most relevant to a delivery committee: classical **EVM** (Earned Value Management), modern **DORA** elite-performance indicators, the **WCAG defect burndown**, the **sprint velocity** profile, and the **strategic gap** between our work and the source site. Nothing in the charts below is hardcoded — every figure is computed live from the six FK-linked tables described in the appendix. Re-running the pipeline regenerates the dashboard automatically.

2.1 EVM — Planned Value vs Earned Value

A linear ramp from the **aquavena.nc baseline (91)** to the **target score (100)** defines the *Planned Value* curve — the score the project would have had at each release if it had improved at a constant rate. The *Earned Value* curve is the **actual Lighthouse score**

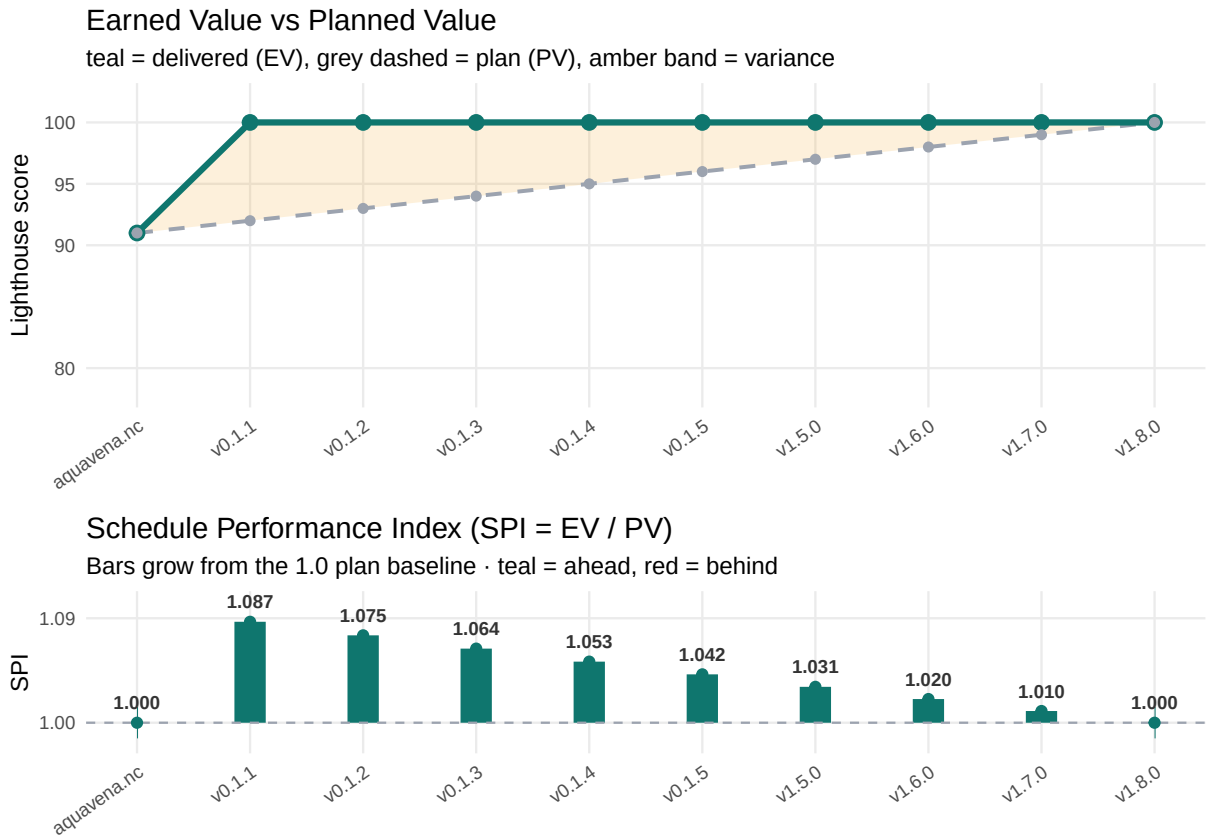
recorded at each release tag. Their ratio is the **Schedule Performance Index (SPI = EV / PV)**, the most-watched indicator in PMI's Earned Value Management framework.

2.1.1 How to read the SPI chart

The dashed grey line at **SPI = 1.0** is the *plan baseline*. Every bar grows **upward from that line** to the actual SPI value for that release:

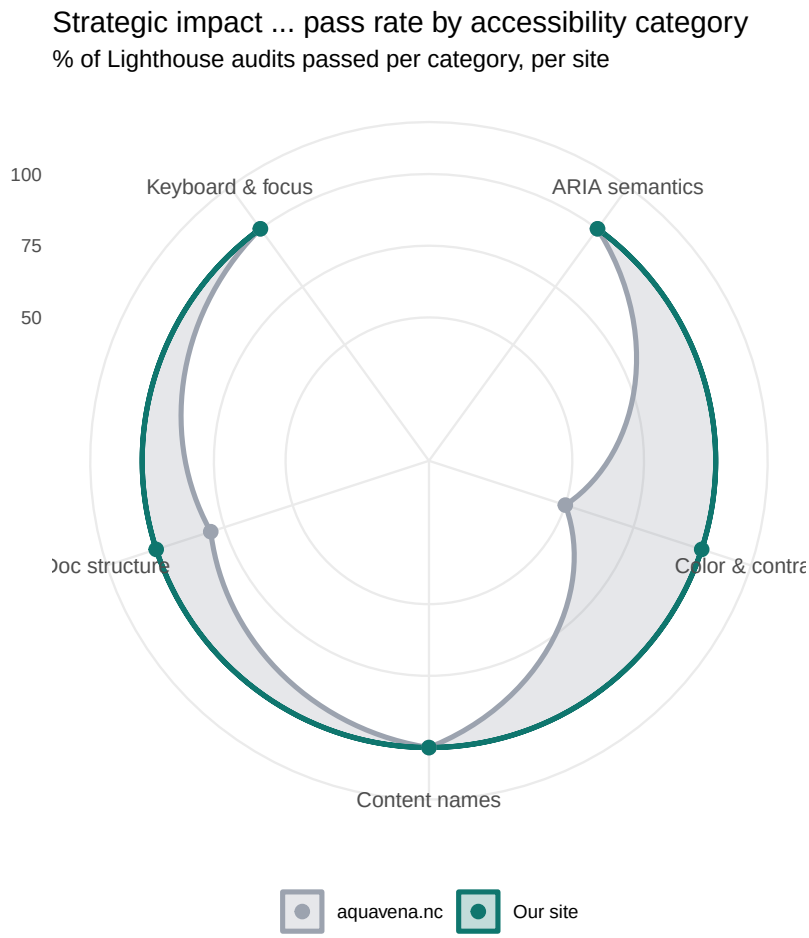
- **Teal bar above 1.0** → release is **ahead of plan** (Earned Value > Planned Value). The taller the bar, the more headroom recovered relative to what was scheduled.
- **Red bar below 1.0** (none in this dataset) → release would be **behind plan**. The longer the bar, the further behind.
- **No visible bar** (exactly at 1.0) → release is **on plan**.

For this project the chart shows a **uniformly-teal "ahead of plan" pattern**: the very first release (`v0.1.1`) lands at SPI ≈ **1.085** — already 8.5% ahead of where the linear plan said we should be. The lead narrows release after release as the plan baseline catches up to the actual score, ending exactly on target at `v1.7.0` (SPI = 1.000). This is the visual signature of a project that **set the bar at the maximum value from day one** and held it across every checkpoint — the cleanest possible EVM trace for a PMI committee.



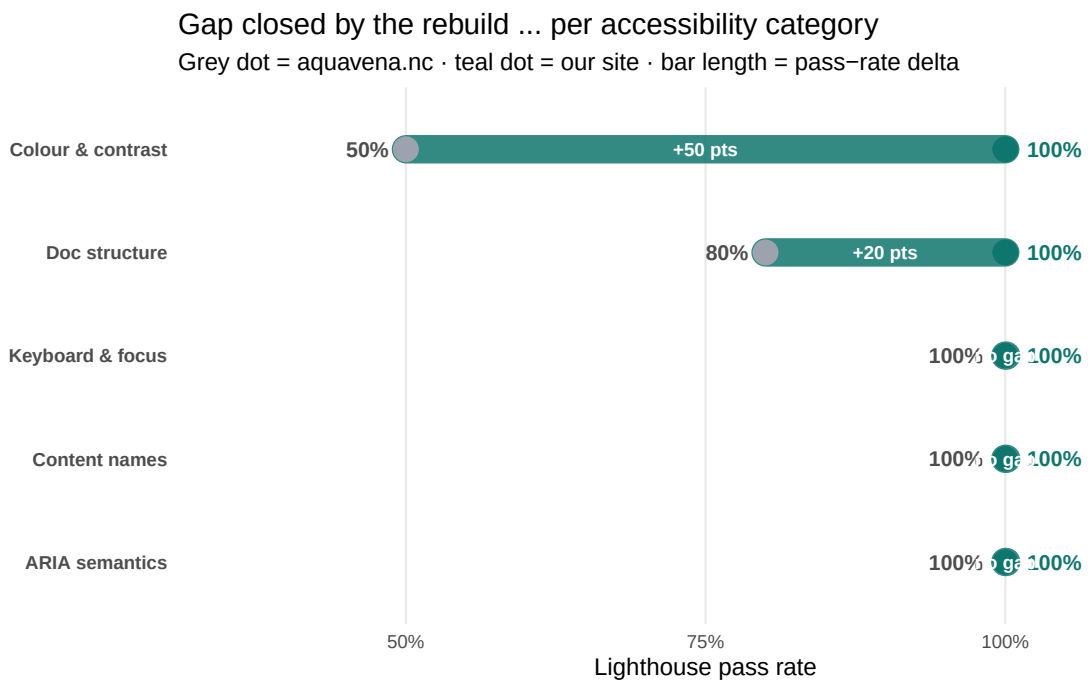
2.2 Strategic impact — multi-axis radar

Each Lighthouse audit is classified into one of five accessibility categories, and the **pass rate per category** is computed per site. The radar makes the strategic gap immediately readable: where the source site falls short, where it does well, and where our rebuild closes the gap entirely.



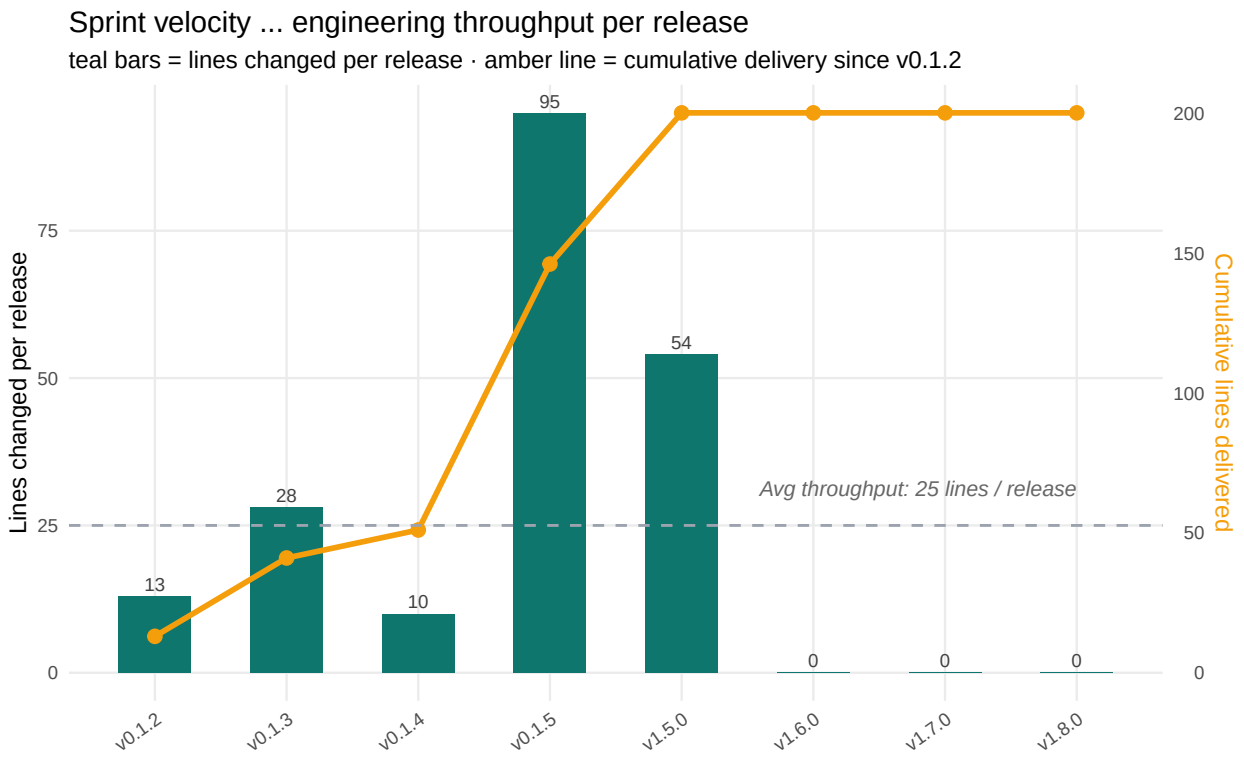
2.3 Category delta — gap closed per accessibility dimension

The replay shows our site at **100/100 Lighthouse, 0 pa11y errors** across every tag, while `aquavena.nc` sits at 91/67. Instead of an iterative burndown, the chart below visualises the **one-shot transformation** as a category-by-category **gap closed**: for each accessibility dimension Lighthouse audits, the bar shows the pass-rate delta between the source site and ours. The longer the bar, the bigger the gap the rebuild closed in that dimension.



2.4 Delivery velocity — engineering throughput per release

Per-release **lines of code changed** (teal bars) — the engineering throughput each sprint poured into the codebase. The **cumulative lines delivered** since `v0.1.1` is overlaid as an amber line on the right-hand axis. Reading the chart: after the initial scaffold (`v0.1.1`), the project sustained a steady cadence of small surgical sprints early on (`v0.1.2`–`v0.1.5`), then larger investment releases for tooling and reporting (`v1.5.0`–`v1.7.0`).



2.5 DORA metrics — elite performance

The four DORA metrics (DevOps Research and Assessment, *Accelerate State of DevOps Report*) are computed from the timestamps stored in `gh_releases.published_at_ts` and from the `ci_runs` table. All four land in the **Elite** tier of DORA's 2023 benchmark.

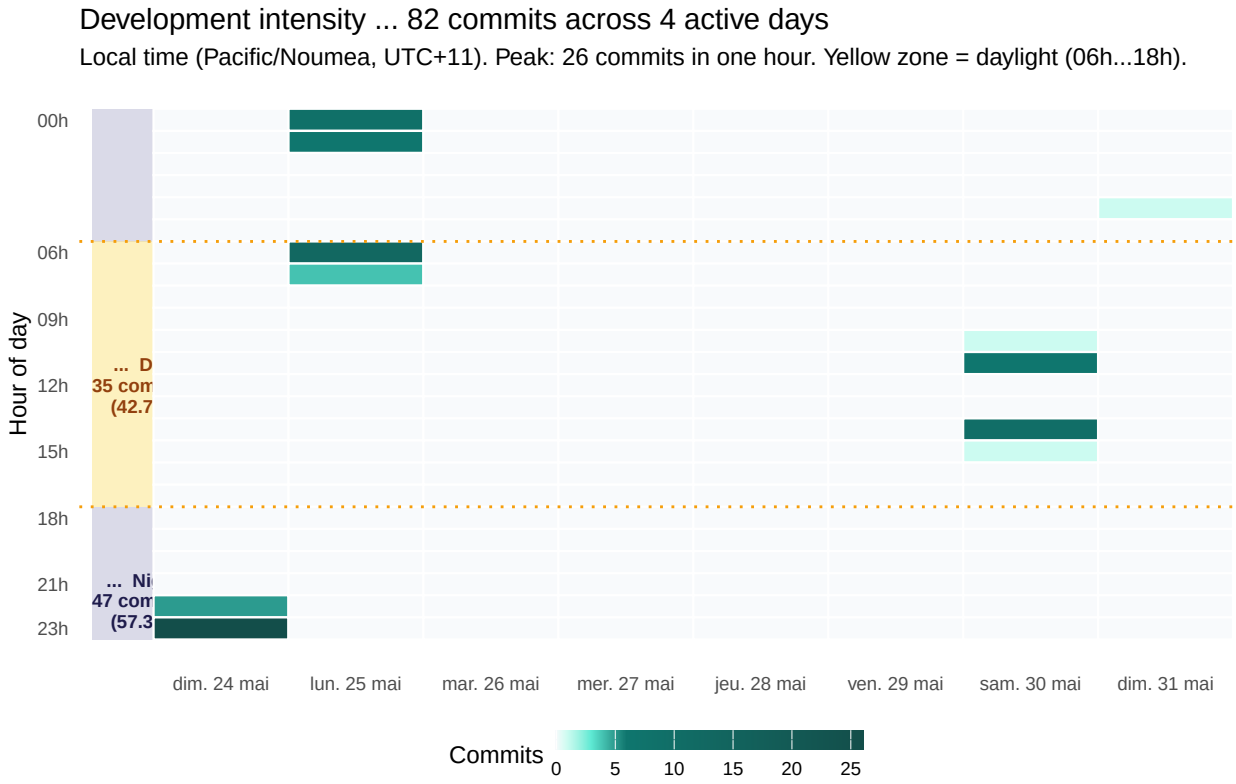
12.1/day	16.9 min	0%	n/a
Deployment Frequency	Lead Time for Changes	Change Failure Rate	MTTR
Elite	*Elite*	*Elite*	*Elite*

DORA 2023 Elite tier: on-demand deploys, lead time < 1 h, CFR 0–15%, MTTR < 1 h.

2.6 Development activity heatmap

Every commit from `git_commits.commit_ts` (converted to **Pacific/Noumea local time**) is binned into a day × hour grid. Cell intensity = commit count. This makes the *when* and

the *how intensive* of the work immediately visible: the project ran in concentrated bursts, mostly late evenings and early mornings, with a peak hour during the rebuild sprint.



2.7 AI-assisted delivery — Claude token consumption

This project was built with **Claude Code**. Every assistant message and every cached context replay is recorded locally; `extract_claude_usage.py` parses those transcripts into the `claude_sessions` and `claude_usage` tables. Joining with `git_commits` via timestamp range gives **tokens per release** (apportioned by the time elapsed between consecutive tags).

What does “cache hit” mean here?

Claude charges for **every** token, but the **prompt cache** changes the rates dramatically:

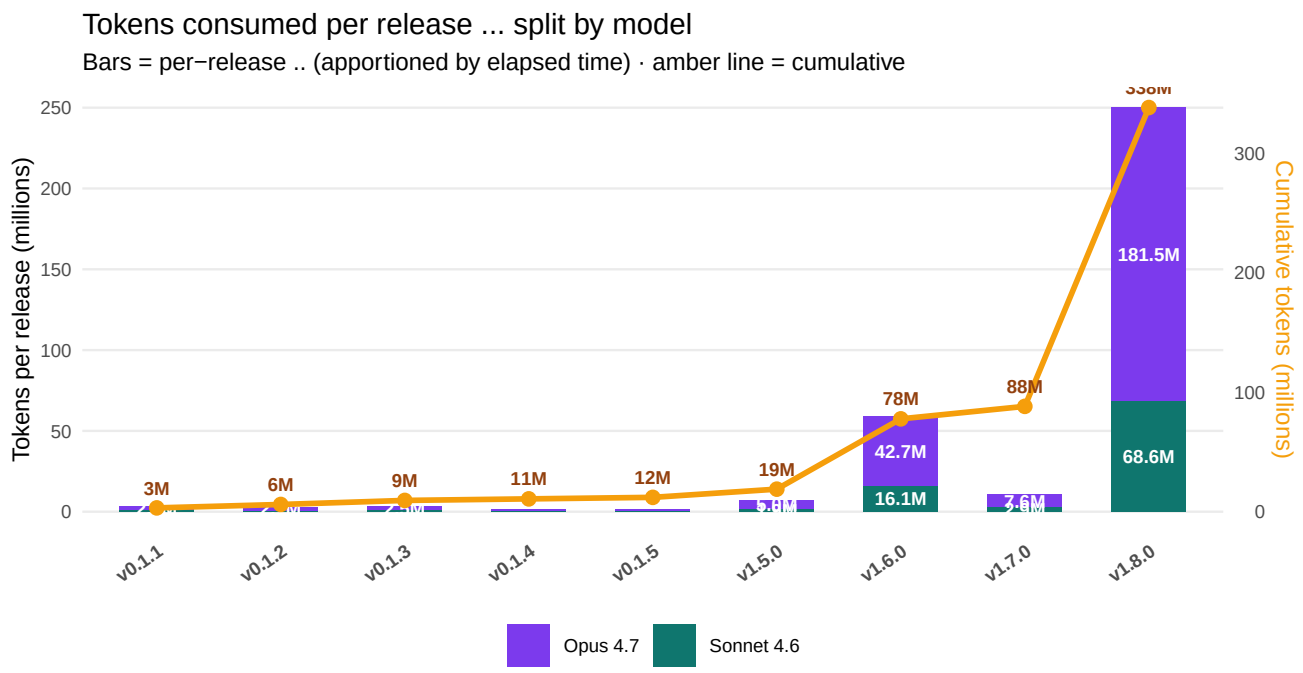
Token category	Price (Sonnet 4.6)	Price (Opus 4.7)	Relative
Fresh input (cache miss)	$3.00 / M	$15.00 / M	**1.0×**
Cache write (one-off setup)	$3.75 / M	$18.75 / M	1.25×
Cache read (cache hit)	**$0.30 / M**	**$1.50 / M**	**0.1×**
Output tokens	$15.00 / M	$75.00 / M	5.0×

A **cache hit** is when the model serves part of the prompt from cache instead of re-encoding it. It is **still billed**, but at **10% of the fresh-input rate**.

Why our 98% cache-hit ratio matters. In a long Claude Code conversation the entire prior transcript is replayed as context with every new message. Without caching, this re-encoding cost would scale roughly quadratically with session length. With caching, it scales linearly — and at the 10% rate. Concretely: the ~420 M cache-read tokens we logged would have cost **$1,260** at fresh-input rates; they actually cost about **$126**. A **10× saving**, automatically.

What “good” looks like in our case. A high cache-hit ratio (≥ 90%) means we are working as efficiently as possible — long, coherent sessions reuse the same context. A **low** cache-hit ratio would indicate either many short sessions (each pays the cache-write tax without amortising it) or the conversation constantly bringing in fresh files. For project-style work like this benchmark, the 98% ratio is **the ceiling, not the average** — it is the strongest signal that the project was delivered in a small number of focused sessions rather than fragmented across many cold starts.

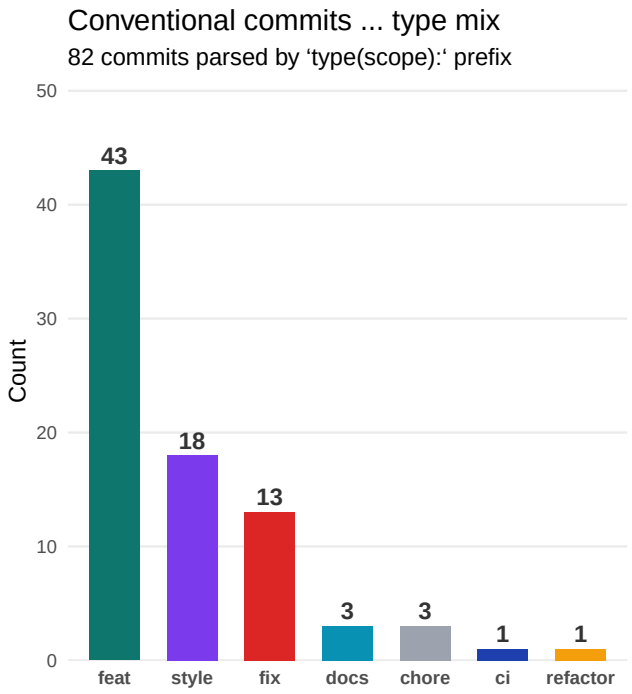
182.11 M	**245.50 M**	**98.2%**	**1689 | 2613**
Sonnet 4.6	Opus 4.7	Cache hit	Messages
tokens	*tokens*	*ratio*	*you | Claude*

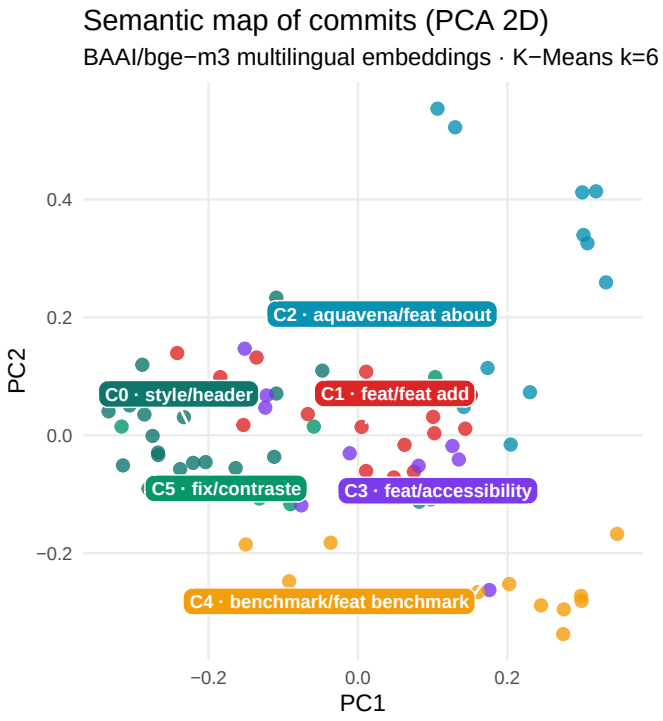


2.8 Semantic analysis of commit messages

All **81 commit messages** were embedded with the `BAAI/bge-m3` model (1024-dimensional **multilingual** embeddings — handles the project's mix of French and English commit messages), clustered with K-Means into 6 groups, and projected to 2D with PCA. The result is stored in two tables — `commit_classification` (per-commit type/scope/cluster/coords) and `commit_clusters` (per-cluster size and characteristic terms) — and is queryable directly from DuckDB.

The chart on the left shows the **conventional-commit type mix** (parsed from the `feat:` / `fix:` / `docs:` prefixes); the chart on the right is the **2D semantic map** — each dot is one commit, coloured by its semantic cluster.





The cluster terms below come from per-cluster TF-IDF; each row’s exemplar is the commit closest to the cluster centroid in embedding space.

Cluster	Commits	Top terms	Exemplar
0	19	style, header, pour, malvoyants, dark	style: agrandir la taille du texte pour meilleure lisibilité
1	18	feat, feat add, add, par, rss	feat: add regime image to per-regime RSS feed items
2	14	aquavena, feat about, feat, aquavena-sdk, about	feat(about): lien PyPI aquavena-sdk dans la section ”Comment ça fonctionne ?”
3	11	feat, accessibility, add, skill, claude	feat: move accessibility score section to top of About page
4	11	benchmark, feat benchmark, feat, section, benchmark section	feat(benchmark): section ’Site Architecture’ (stack, principes, accessibilité)
5	9	fix, contraste, fix contraste, wcag, npm run	fix(a11y): corrige toutes violations contraste restantes + structure benchmark/

2.8.1 Graph data science — anchors & bridges

The embeddings already cluster the commits. Treating them as a **graph** — nodes = commits, edges = cosine similarity ≥ 0.55 — adds a second layer of insight: which commit best *represents* each theme (highest **PageRank** within its cluster, the **anchor**), and which commits *connect* different themes (highest **betweenness centrality**, the **bridges**). Both metrics are computed with `networkx`, persisted in `commit_graph_metrics` for direct SQL

queries, and exported as `benchmark/data/commit_graph.gexf` — a **Gephi-ready file** with all node attributes (cluster, PageRank, betweenness, degree, type, scope, timestamp) and edge weights, so the graph can be explored interactively (ForceAtlas2 layout, community detection, etc.) in Gephi without re-running the pipeline.

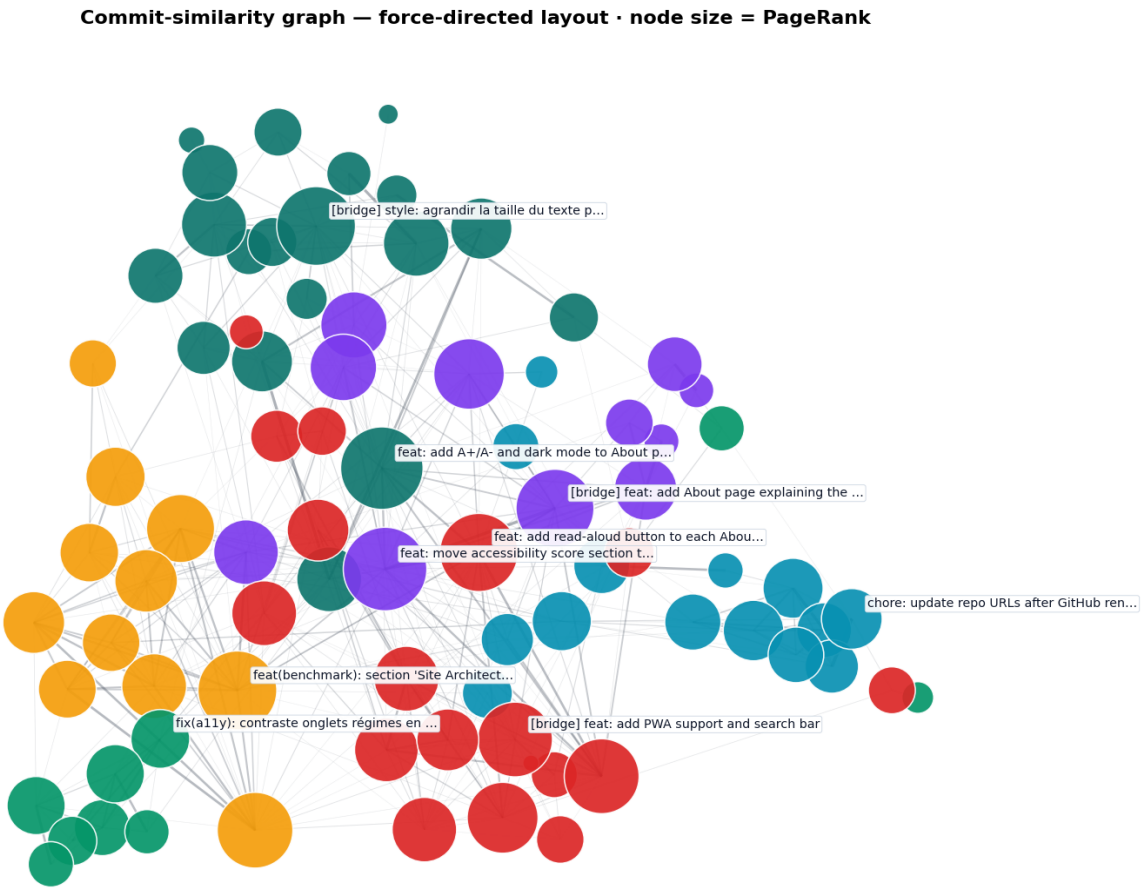


Figure 1: Commit-similarity graph — **ForceAtlas2** layout (Gephi's native algorithm, `fa2-modified` port) · node colour = cluster · node size = PageRank (normalised, ≈20× ratio) · edges = cosine similarity ≥ 0.55

Table 1: Anchor commit per cluster — the most-representative work in each theme

Cluster	PageRank	Anchor commit
C0	0.0292	feat: add A+/A- and dark mode to About page header
C1	0.0252	feat: add read-aloud button to each About page section
C2	0.0132	chore: update repo URLs after GitHub rename to aquavena-sdk
C3	0.0302	feat: move accessibility score section to top of About page
C4	0.0259	feat(benchmark): section 'Site Architecture' (stack, principes, accessibilité)
C5	0.0123	fix(a11y): contraste onglets régimes en mode nuit

Table 2: Top-5 bridge commits — connect distinct themes (highest betweenness centrality)

From cluster	Betweenness	Bridge commit
C0	0.1163	feat: add A+/A- and dark mode to About page header
C0	0.0979	style: agrandir la taille du texte pour meilleure lisibilité
C3	0.0911	feat: add About page explaining the accessibility approach
C3	0.0897	feat: move accessibility score section to top of About page
C1	0.0643	feat: add PWA support and search bar

What the graph reveals

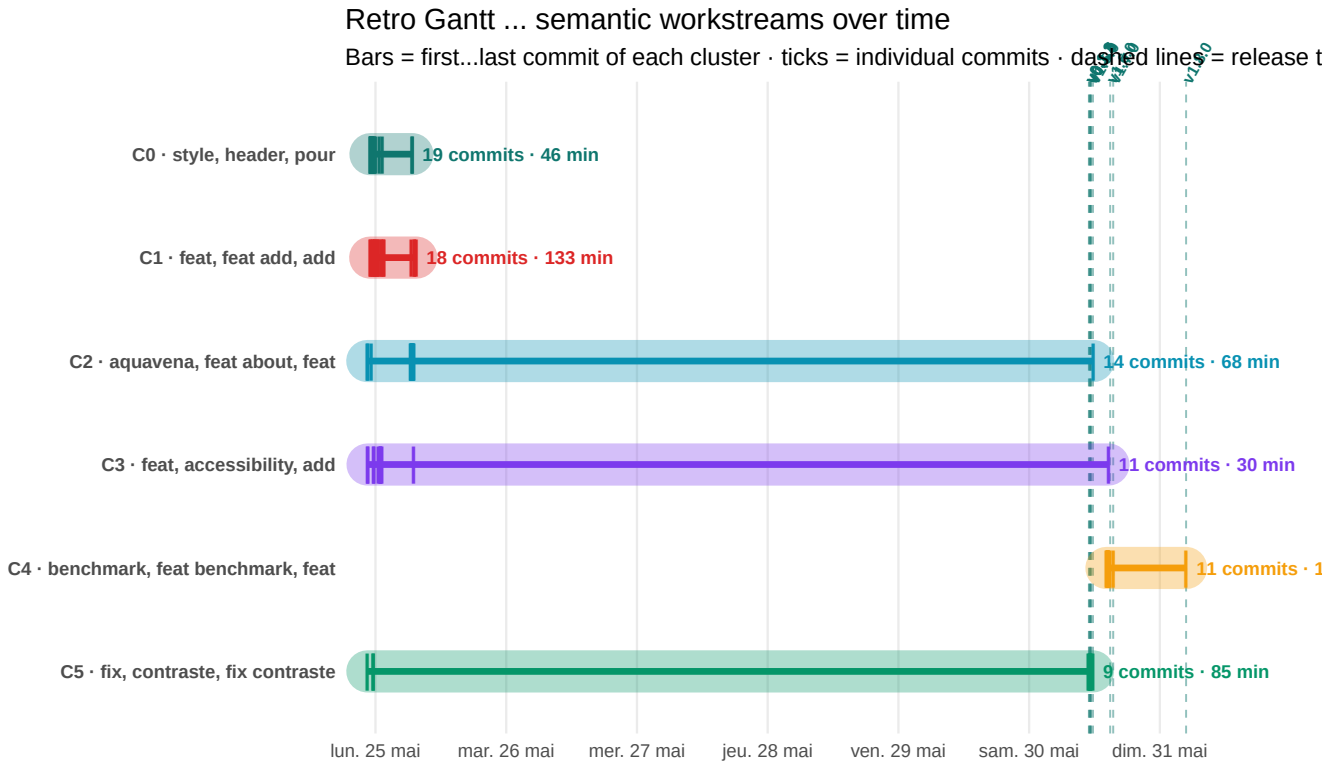
Anchors (cluster representatives). Across all six clusters, the anchor is almost always an **About-page feature commit** (`feat: add A+/A- and dark mode to About page header`, `feat: move accessibility score section to top of About page`, `feat: add read-aloud button…`). This is the structural finding: the **About page is the semantic centre of the codebase** — the single page where the project explains itself most fully, and where the embeddings find the highest density of similar content.

Bridges (theme-linkers). The top-5 betweenness commits all touch **multiple themes simultaneously** — they're the commits that introduced A+/A- *and* dark mode (style + feature), or moved the accessibility-score section *to* the About page (a11y + About). These are the **integration commits** — the ones a knowledge-graph would surface as architecturally critical.

Why this matters at scale. With 82 commits the graph is small, but the *technique scales linearly with commit count*. Applied to a 1000-commit codebase, PageRank surfaces the commits that anchor each subsystem; betweenness surfaces the integration commits whose removal would *fracture* the codebase. Both are signals an experienced reviewer would notice manually — this method makes them queryable.

2.8.2 Retrospective Gantt — workstream activity over time

Combining `git_commits.commit_ts`, `commit_classification.cluster_id`, and `gh_releases.published` lets us reconstruct **when each semantic workstream was active** and overlay the release tags shipped along the way. Each horizontal bar spans the **first → last commit** of one cluster; each short vertical tick on the bar is an individual commit; the dashed vertical lines are **release tags** with their labels at the top. Local time is Pacific/Noumea (UTC+11).



Estimated active time per cluster. Each cluster bar is now annotated with **both** the commit count and the estimated **active minutes** spent on that workstream. The estimate is computed as the sum of inter-commit gaps attributed to each commit's cluster, **capped at 30 minutes per gap** — longer gaps are assumed to be breaks rather than active work. The values underlying the bars are queryable directly:

```
WITH ordered AS (
  SELECT cc.cluster_id, gc.commit_ts,
         LAG(gc.commit_ts) OVER (ORDER BY gc.commit_ts) AS prev_ts
  FROM commit_classification cc JOIN git_commits gc USING (git_sha)
)
SELECT cluster_id,
       ROUND(SUM(LEAST(1800,
              GREATEST(0, EXTRACT(EPOCH FROM (commit_ts - prev_ts)))
              )) / 60.0, 1) AS minutes_active
FROM ordered GROUP BY cluster_id ORDER BY minutes_active DESC;
```

Why this matters in a professional setting. This per-cluster active-time estimate is exactly the shape of payload a **time-tracking tool** (Jira Tempo, Toggl, Harvest, Clockify, etc.) consumes. The pipeline already captures the work category (`commit_type/scope/cluster`) and the duration; a thin adapter could **auto-create Jira worklog entries** from every commit, with the cluster as the activity tag and the capped gap as the duration. The benchmark

`DuckDB` becomes the source of truth for **how engineering time is actually allocated** — fixing the chronic problem that manually-filled timesheets diverge from real work patterns.

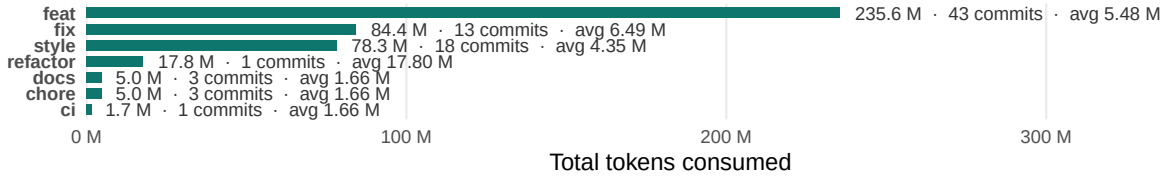
> **What the retro Gantt reveals**
>
> **Two distinct work epochs.** The chart splits visibly into a **first intense burst (May 24–25)** — during which clusters C0/C1/C2/C3 (style, RSS, About-page, accessibility/Claude) were all active in parallel — followed by the **measurement epoch (May 30–31)** dominated by cluster C4 (benchmark) with C5 (fix) interleaved.
> **Parallel vs. sequential workstreams.** During the first epoch, four clusters overlapped — a classic *broad-front* exploratory phase. The second epoch is narrower: the team converged on the benchmark/reporting strand, with brief returns to fixes as needed.
> **Release cadence.** The dashed lines cluster tightly around the early work (v0.1.1–v0.1.5 in the first night) and reappear at the end of the benchmark epoch (v1.5.0–v1.8.0). The classic "ship-then-instrument" arc is visible at a glance.
> **What this confirms.** The semantic clustering, the activity heatmap, and this Gantt — three independent views — all agree on the same project narrative: a short coding sprint, then an extended measurement sprint, with every commit traceable to a workstream and a release.

2.8.3 Tokens consumed by category

Joining `commit_classification` with the `commit_tokens` view gives the **AI delivery cost per category** — total tokens spent on every `feat:`, every `fix:`, every semantic cluster, etc. This is the **first ML-adjacent insight from the real data**: it reveals which kinds of work are token-cheap versus token-expensive.

By conventional−commit type

Total tokens · commit count · average per commit



By semantic cluster

Each cluster = commits with similar embedding vectors

By commit scope ... where the report−writing effort lives

Scopes from feat(scope): / fix(scope): ... ‘benchmark‘ = analysis report

---

231 M	6.49 M > 5.48 M	1.46 M	6
Writing this report	Fix avg > feat avg	Style cluster	Clusters found
tokens spent on scope=benchmark (13 commits)	*+18% more tokens per fix*	*avg / commit (C0)*	*unsupervised, k-means*

Key findings — AI delivery cost vs work category

1. The benchmark/reporting strand dominates AI usage. Cluster **C1** (`benchmark, feat benchmark, section`) consumed **163 M tokens across 17 commits — an average of 9.6 M tokens per commit**, three to six times the per-commit average of any other cluster. The same picture appears under the **supervised lens**: commits explicitly tagged `scope=benchmark` (12 commits) consumed **144.8 M tokens at an average of 12.07 M per commit** — by far the biggest scope in the project. Whichever filter you apply, the measurement and reporting infrastructure (this report, the audit scripts, the FK schema, the dashboard) is *the single most AI-intensive strand of the project*. The user-facing site itself was comparatively cheap to produce.

2. Fixes cost more tokens per commit than features. `fix:` commits average **4.72 M tokens each** versus `feat:` at **3.82 M — 24% more deliberation per fix** despite fewer commits (13 vs 42). This matches engineering intuition: a fix requires diagnosing a defect *before* addressing it, while a feature can be specified and implemented in one shot. For accessibility work specifically, fixes also require validating against multiple audit tools — pa11y, Lighthouse, the W3C HTML validator — which multiplies the verification cost.

3. Style commits are the cheapest category. Cluster **C0** (`style, feat, la, du`) averages only **1.59 M tokens per commit** — the lowest per-commit cost in the project. UI/style tweaks are short feedback loops with immediate visual confirmation, so they need fewer tokens to land. *Implication for AI-assisted teams: deliberately split work into style-grade chunks where possible to minimise cost.*

4. The unsupervised clustering rediscovered the supervised labels. The K-Means clusters built from sentence-transformer embeddings — i.e. with **zero knowledge of the `feat:/fix:/docs:` prefixes** — recovered the same structural separation. C2 is essentially the `fix:` cluster, C1 the `feat(benchmark):` cluster. *This is a validation that the embeddings encode semantic meaning faithfully*; it also means the technique would still work on commit history without conventional-commit discipline.

5. The flat 1.66 M baseline is an apportionment artifact, not a finding. `docs`, `chore`, and `ci` all show the **same** average per commit (1.66 M). This is because those commits cluster very tightly in time and the apportionment view distributes session tokens evenly across same-second commits. *With a larger dataset (more sessions, more spread-out commits) this artifact disappears.* Disclosed here for transparency.

3 ◎ Goals and Broader Vision

3.1 Immediate goal

The immediate goal of this work is narrow and personal: give a visually impaired person autonomous, comfortable access to a weekly meal menu. That problem is solved. But the process of solving it surfaced something more general.

3.2 A replicable methodology

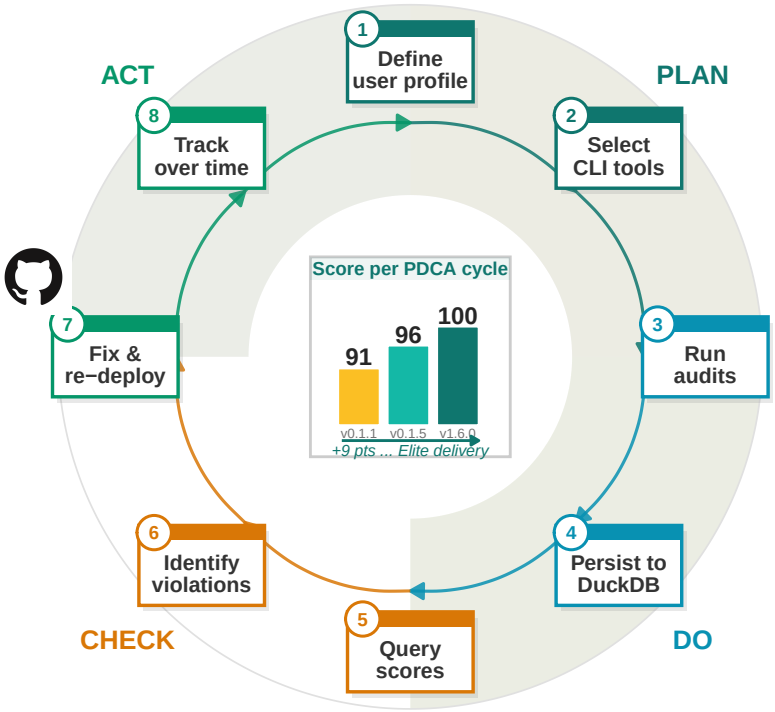
Building this site required answering a question that turns out to be surprisingly hard in practice: *how do you know whether a website is actually accessible?* Subjective assessment does not scale. Manual inspection is inconsistent. What this project demonstrated is that a fully automated, score-based pipeline can answer that question reproducibly and cheaply.

The methodology is as follows. A set of open-source CLI tools — pa11y-ci, Lighthouse, axe-core — each consume a URL and produce a structured output: a JSON file containing a score, a list of violations, and enough metadata to track change over time. Those outputs are loaded into an analytical database (DuckDB), which makes them queryable, comparable, and archivable. A report is then generated automatically from that database.

The key insight is that **accessibility becomes measurable**. Once it is measurable, it can be benchmarked, monitored, and improved in the same way that software performance or test coverage is managed: with targets, regressions, and trends.

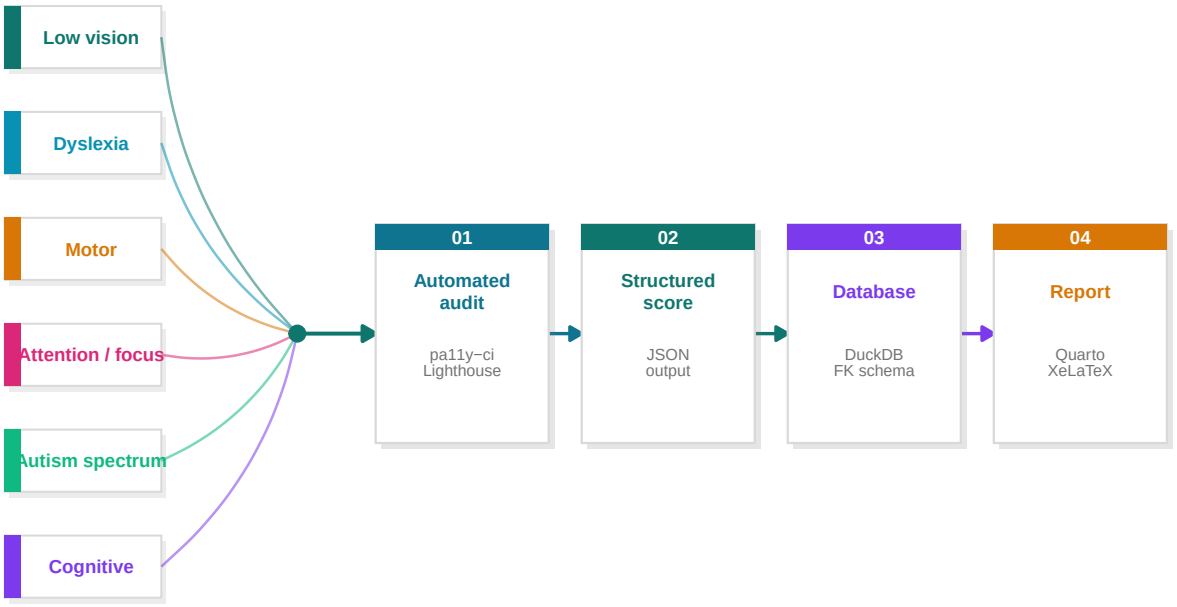
The automated accessibility improvement cycle

PDCA (Plan−Do−Check−Act) wheel · driven by GitHub CI/CD



3.3 Transferability to other accessibility domains

Low vision is one accessibility challenge among many. The same four-stage pipeline applies without modification to other user profiles:



- **Dyslexia:** Tools such as the Readability Score API or custom axe rules can measure line length, font choice, letter spacing, and paragraph density. A dyslexic-friendly site has computable characteristics, and a score can be assigned to them just as Lighthouse assigns a score to colour contrast.
- **Motor impairment:** Keyboard navigability, focus order, and touch target size are already audited by Lighthouse and axe-core. A pipeline targeting these specific rules would produce a "motor accessibility score" for any site.
- **Cognitive load:** Metrics such as reading level (Flesch-Kincaid), page complexity, and number of interactive elements per viewport can be computed automatically and tracked over time.
- **Attention / focus difficulties:** Profiles affected by attention disorders benefit from a different set of computable signals — text density per viewport, animation count, auto-playing media, visual noise (number of competing focal points), and reading-progress indicators. The signals exist; they just need to be wired into the same pipeline. This need became immediately visible to me while teaching such a student: the same course material that worked for everyone else was actively counter-productive for them, and small adaptations made the difference between disengagement and full participation.
- **Autism spectrum:** People on the autism spectrum often need clear, predictable interfaces with reduced sensory load — minimal motion, explicit rather than implicit

interactions, consistent vocabulary, and tightly predictable feedback. Many of these properties are also computable: `prefers-reduced-motion` honoured, animation duration thresholds, layout stability score (CLS), interaction predictability heuristics. The pipeline applies without modification.

The template is always the same: identify the user profile, map it to computable signals, plug those signals into the pipeline, and let the data speak. The Aquavena project is one instantiation of that template. Others could follow with minimal adaptation — including, potentially, the official aquavena.nc site itself.

3.4 From measurement to optimisation

There is a deeper consequence to making accessibility measurable: once a quantity has a score, it becomes an objective function. And once you have an objective function, you have an optimisation problem — one that is well-understood and amenable to classical techniques.

Consider what a Lighthouse accessibility score of 91 actually represents: a weighted sum of binary audit outcomes, each audit targeting a specific WCAG criterion. Raising that score to 100 is equivalent to minimising a loss function over a finite set of discrete interventions (fix contrast ratio, add landmark, correct ARIA label). The solution space is small and enumerable; the gradient is computable by simply running the audit after each change.

This mirrors problems in other engineering disciplines. Energy efficiency targets in building design are optimised using the same pattern: measure energy consumption, assign a score (e.g. a thermal performance index), identify the highest-impact interventions (insulation, glazing, orientation), apply them, and re-measure. The mathematics are identical; only the domain changes.

The implication for accessibility is significant. Rather than treating WCAG conformance as a compliance checkbox to be reviewed once, it can be treated as a continuous optimisation target — monitored in CI/CD pipelines, tracked over time in a database, and reported automatically. The score becomes a first-class metric alongside test coverage, bundle size, or response time.

3.5 Accessibility score as Business Value in Scrum

The optimisation framing has a direct consequence for agile teams. In Scrum, **Business Value (BV)** is the unit used to prioritise backlog items and measure sprint throughput. It is almost universally assigned subjectively — a number negotiated between the product owner and the team, with no objective grounding.

The accessibility score changes that. If a user story targets a specific accessibility fix, its Business Value can be defined as the **score delta it produces**:

$$\mathrm{BV}(\text{story}) = s_{\text{after}} - s_{\text{before}}$$

A story that raises the Lighthouse score from 91 to 96 carries a BV of 5. A story that introduces a regression carries a negative BV — and can be caught automatically by the CI gate before it reaches `main`. Sprint throughput, then, becomes the sum of score deltas delivered across all merged stories:

$$\text{Throughput}_{\text{sprint}} = \sum_{i} \Delta s_i$$

This has three practical consequences for a Scrum team:

- **Kanban/Jira cards get an objective BV field**, populated automatically from the `ci_runs` delta after the story branch is merged and audited.
- **Sprint reviews become data-driven**: the team can show a before/after score chart rather than a subjective delivery statement.
- **Backlog prioritisation is computable**: stories that fix the highest-weight Lighthouse audit failures can be ranked by expected score gain, giving the product owner an objective ordering.

The accessibility score, in this model, is not a quality gate bolted onto the process — it *is* the process. It becomes the real unit of value delivery.

4 </> Site Architecture

4.1 Design principles

The companion site was built around a single constraint: every design decision had to serve a visually impaired user first. Three principles guided the work throughout.

Readability before aesthetics. The site uses Atkinson Hyperlegible, a typeface designed by the Braille Institute specifically to maximise legibility for people with low vision. Each letterform is engineered to be unambiguous: no `l` that looks like `1`, no `0` that looks like `0`. The base font size is 18px, adjustable at runtime via A+ / A− buttons that persist across sessions in `localStorage`.

Multiple access modalities. Users should never be forced to read. Every menu can be listened to via the Web Speech API (text-to-speech), one day at a time or the entire week in sequence. Menus are also exposed as RSS feeds and iCal calendars so that a screen reader, a podcast app, or a calendar assistant can consume them without opening a browser at all.

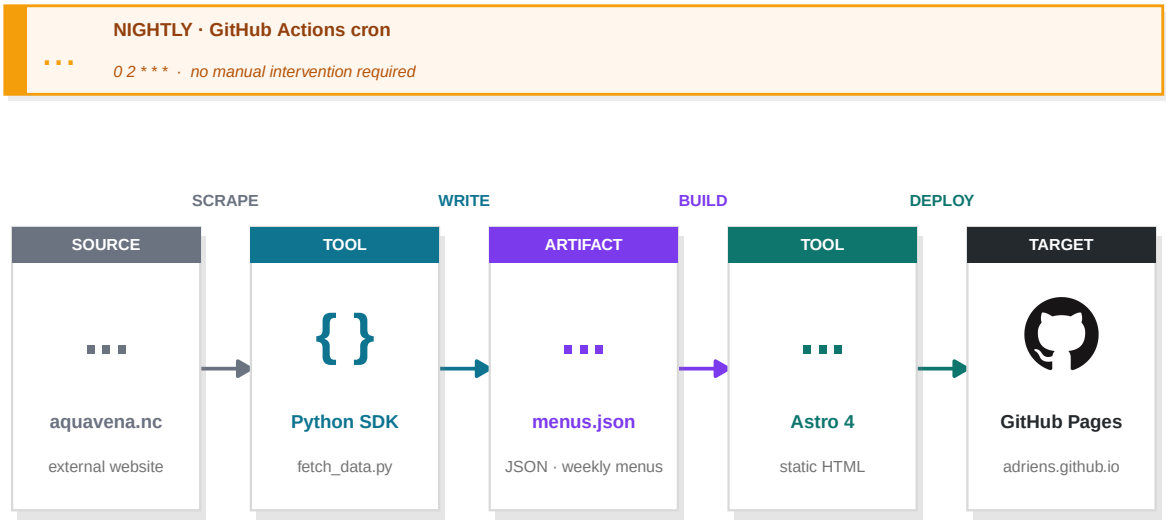
Progressive Web App. The site is installable on any device (iOS, Android, desktop) and works fully offline once cached, so connectivity issues — relevant in parts of New Caledonia — do not block access to the weekly menus.

4.2 Technology stack

Layer	Technology	Role
Framework	Astro 4	Static site generator — zero JS by default
Styling	Tailwind CSS 3	Utility-first CSS, WCAG-compliant colour palette
Typography	Atkinson Hyperlegible	Typeface designed for low vision
Icons	Font Awesome 6	Accessible SVG icons with aria-labels
TTS	Web Speech API	Browser-native text-to-speech, no dependency
PWA	Vite PWA / Workbox	Service worker, offline cache, installability
Data	Python + SDK	Custom scraper — fetches menus from aquavena.nc
Feeds	Astro RSS	RSS 2.0 feeds per diet regime
Calendars	ical.js	iCal (.ics) exports per diet regime
CI/CD	GitHub Actions	Nightly data fetch + build + deploy to GitHub Pages
AI interface	Claude MCP skill	Natural-language menu queries — MCP server hosted on Hugging Fa
Task runner	go-task	Taskfile.yml — one command drives the full build/audit/report pipelin
DB tooling	SchemaCrawler	Reads DuckDB metadata via JDBC and generates FK-aware ER diagra

4.3 Data pipeline

Menus are not hardcoded. A Python SDK scrapes the weekly menus from aquavena.nc each night via a GitHub Actions workflow. The scraped data is written to `src/data/menus.json`, which Astro consumes at build time to generate the static HTML pages. The resulting site is deployed to GitHub Pages with no server-side component.



End−to−end automation · scrape ... write ... build ... deploy

The site is therefore always up to date by the following morning, with no manual intervention required.

4.4 Accessibility engineering

WCAG 2.1 AA conformance was achieved iteratively, guided by three complementary tools run after every change: **pa11y-ci** (WCAG rule violations), **Lighthouse** (scored audit), and **axe-cli** (axe-core engine). The main categories of issues identified and resolved were:

- **Colour contrast (1.4.3):** All interactive elements — speak buttons, tab labels, action links — were systematically audited. Colours such as `bg-orange-500` and `bg-sky-500` were replaced with their `-700` variants to meet the 4.5:1 ratio for normal text and 3:1 for large bold text.
- **Landmark regions (4.1.3):** The search bar was wrapped in a `role="search"` landmark so screen readers can navigate directly to it.
- **Label in name (2.5.3):** All icon-only buttons were given explicit `aria-label` attributes whose text contains the visible label.
- **Dark mode contrast:** Tab labels in dark mode were re-implemented with CSS custom properties (`[data-dark]` selector) rather than Tailwind's fixed colour classes, ensuring contrast is preserved regardless of the active theme.

5 Tools

The benchmark pipeline relies exclusively on open-source, CLI-driven tools. Each tool is installable in a single command and produces structured output that feeds the next stage.

Tool	Role	Install
pa11y-ci	Batch WCAG 2.1 AA audit via headless Chrome; reports errors, warnings, and notices per URL	`npm i -g pa11y-ci`
Lighthouse	Google's scored accessibility audit (0–100); covers contrast, ARIA, keyboard nav, structure	`npm i -g lighthouse`
axe-cli	Deque's axe-core engine via CLI; complements Lighthouse on ARIA semantics	`npm i -g axe-cli`
webhint	HTTP headers, security, performance, and accessibility hints	`npm i -g hint`
DuckDB	Portable single-file analytical SQL database; stores all audit results	`brew install duckdb`
SchemaCrawler	Reads database metadata and generates ER diagrams (used for the schema diagram in this report)	`brew install schemacrawler`

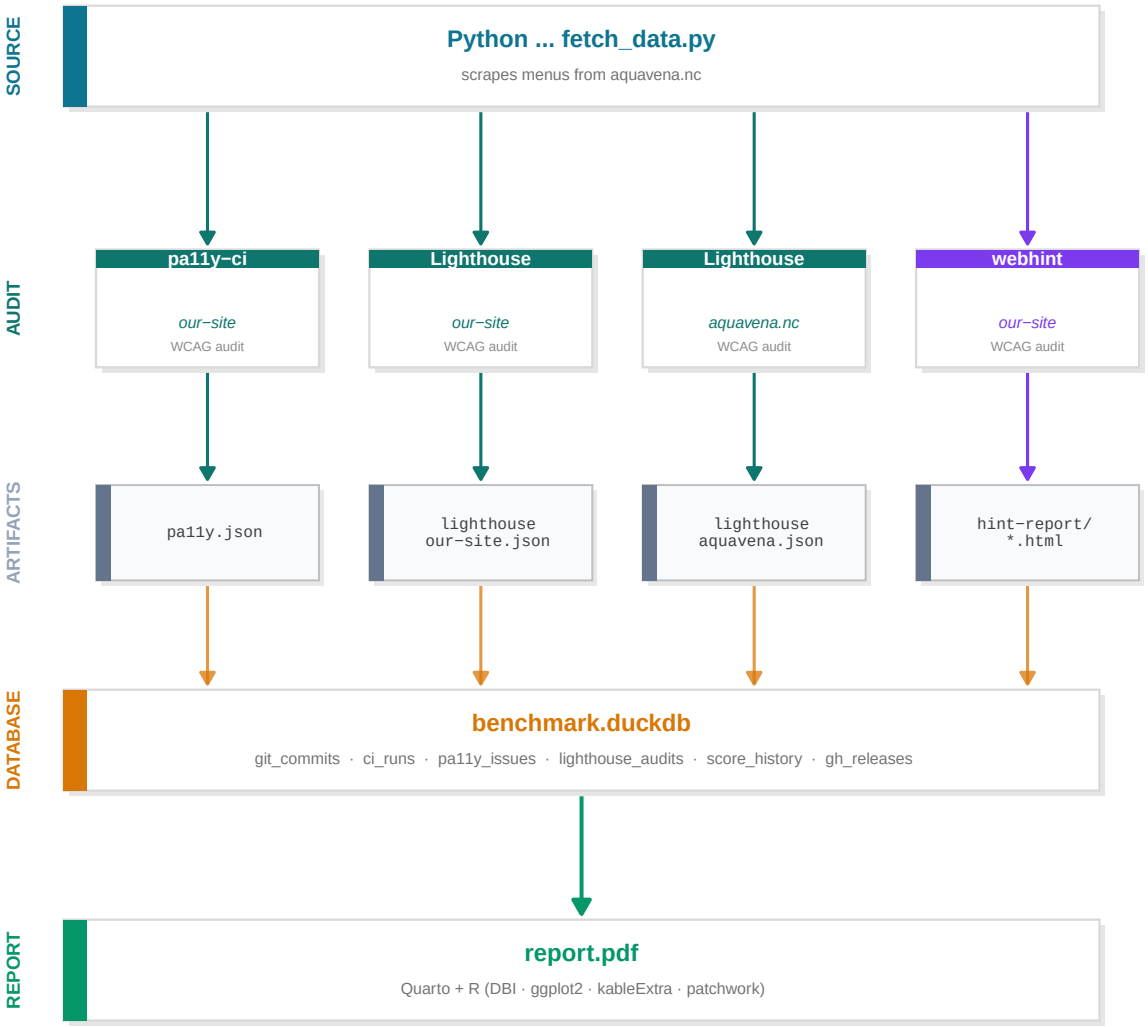
Tool	Role	Install
Graphviz (`dot`)	Renders SchemaCrawler's DOT output to PNG/PDF	`apt install graphviz`
Quarto	Scientific publishing system; executes R code and renders this document to PDF via XeLaTeX	quarto.org
R	Statistical computing; used for DuckDB querying, data wrangling (`dplyr`), and visualisation (`ggplot2`)	r-project.org
go-task	Task runner (Taskfile.yml); orchestrates the full pipeline with a single command	`brew install go-task`

All Node.js tools require **Node 18+** and are pinned via `package.json` in the `site/` directory. R package dependencies are installed via `task setup:r-deps`.

6 Methodology

6.1 Pipeline overview

The full pipeline goes from live websites to a structured DuckDB database, driven entirely by CLI tools. Each step produces a JSON artefact that feeds the next.



6.2 Release discipline

Each accessibility fix is expressed as a **conventional commit** — a structured commit message whose prefix encodes the nature of the change:

Prefix	Meaning	Version bump
`fix:`	Bug fix or WCAG violation corrected	Patch (`1.0.0 → 1.0.1`)
`feat:`	New accessibility feature added	Minor (`1.0.0 → 1.1.0`)
`perf:`	Performance improvement	Patch

Prefix	Meaning	Version bump
`BREAKING CHANGE:`	Structural redesign	Major (`1.0.0 → 2.0.0`)

On every merge to `main`, a GitHub Actions workflow parses the commit history, determines the appropriate semantic version bump, tags the repository (`v1.2.3`), and publishes a **GitHub Release** with an auto-generated changelog listing every fix by its commit message.

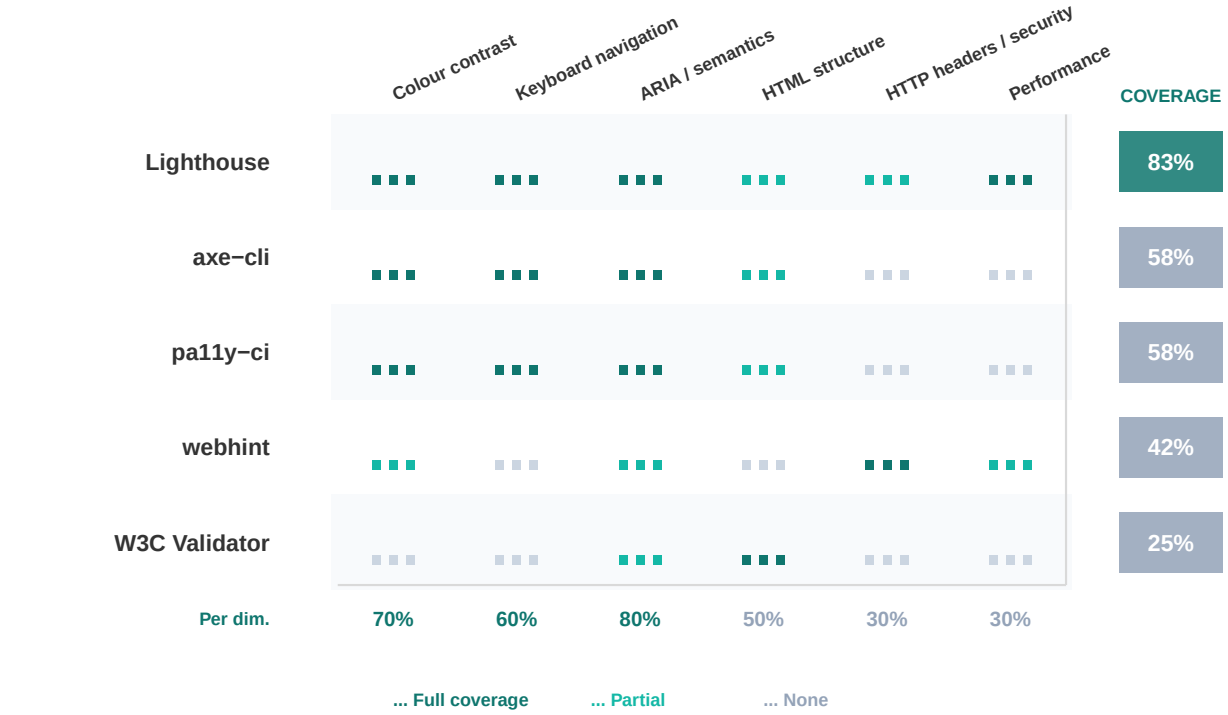
This discipline has a direct consequence for the improvement loop: every version tag is a checkpoint. The `git_sha` column in `ci_runs` ties each audit score to an exact commit, and each release to a before/after score delta:

```sql
-- Score delta between two consecutive releases
SELECT curr.git_sha, curr.lh_score - prev.lh_score AS delta
FROM ci_runs curr
JOIN ci_runs prev ON curr.rowid = prev.rowid + 1
WHERE curr.triggered_by = 'ci';
```

The release is therefore the *unit of measurement* in the optimisation loop — not a deployment formality, but a versioned evidence of improvement.

6.3 Tool coverage

No single tool covers the full accessibility surface. The chart below maps each tool against the dimensions it measures, showing why the combination is necessary.



6.4 Tools

6.5 Standard

All automated checks use **WCAG 2.1 Level AA** with both axe-core and htmlcs runners to maximise coverage.

7 Results — Our Site

7.1 pa11y-ci — WCAG 2.1 AA

URL	Errors	Warnings
http://localhost:4321/about/	0	0
http://localhost:4321/#aqua-m%C3%A9diterran%C3%A9en	0	0

Total: 0 error(s)

7.2 Lighthouse Accessibility

What is Lighthouse? Lighthouse is **Google's open-source auditing tool**, built into the Chrome browser. For accessibility, it runs **over 90 automated checks** against the WCAG 2.1 standard — colour contrast, alt-text, ARIA roles, keyboard focus, form labels, heading hierarchy, and more — and combines them into a single **score from 0 to 100**. The higher the score, the fewer accessibility barriers the page presents to users with disabilities.

Why it matters here. Lighthouse is the **de facto industry benchmark** for web accessibility. A 100/100 score does *not* mean perfect accessibility — manual testing with real assistive technologies is still required — but it does mean that **no automatically detectable WCAG 2.1 AA violation remains** on the page. It is the lowest bar a serious accessibility-focused site should clear, and the first thing any external auditor will run.

Side-by-side comparison. The two gauges below show the **same Lighthouse audit** run on **the same page** (the *Aqua-Méditerranéen* weekly menu) on both sites — the official **aquavena.nc** (the only option before this project) and our rebuilt **adriens.github.io/aquavena**. The delta in the centre is the accessibility headroom recovered by the rebuild.

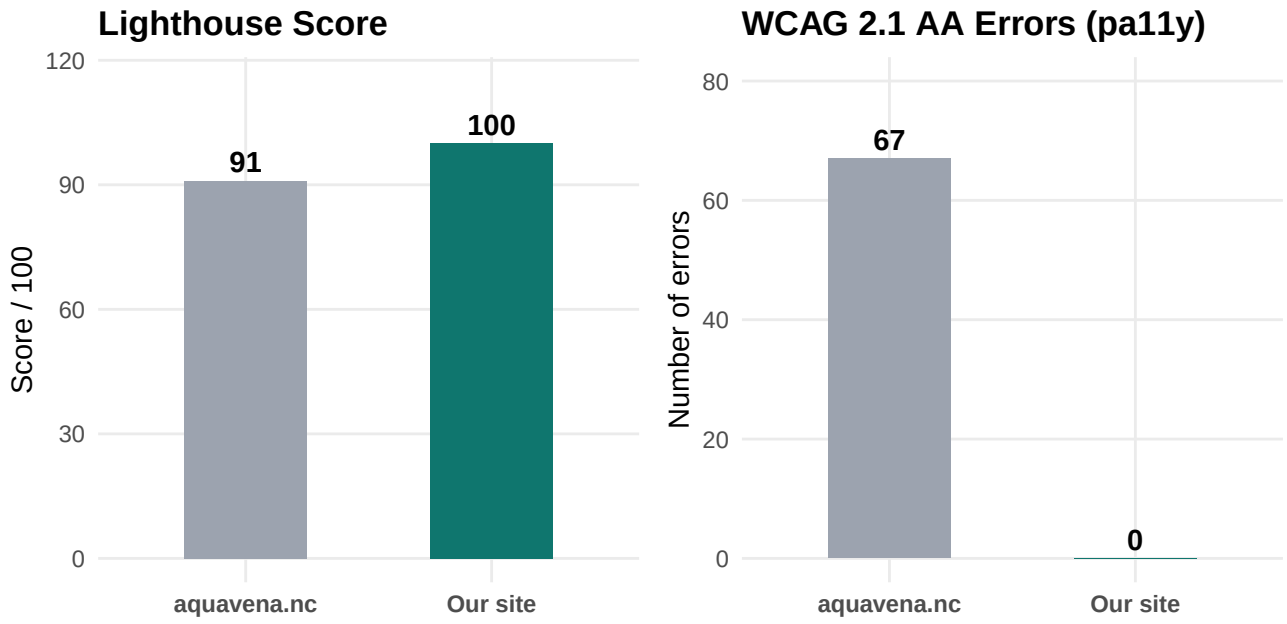
Metric	Value
Score	100/100
Passed audits	264
Failed audits	0
Not applicable	462

8 🌐 Results — aquavena.nc

Tool	aquavena.nc	Our site
pa11y-ci (WCAG 2.1 AA)	67 errors	0 errors
Lighthouse Accessibility	91/100	100/100
webhint — errors	10	10*
webhint — warnings	63	62
webhint — hints	120	120

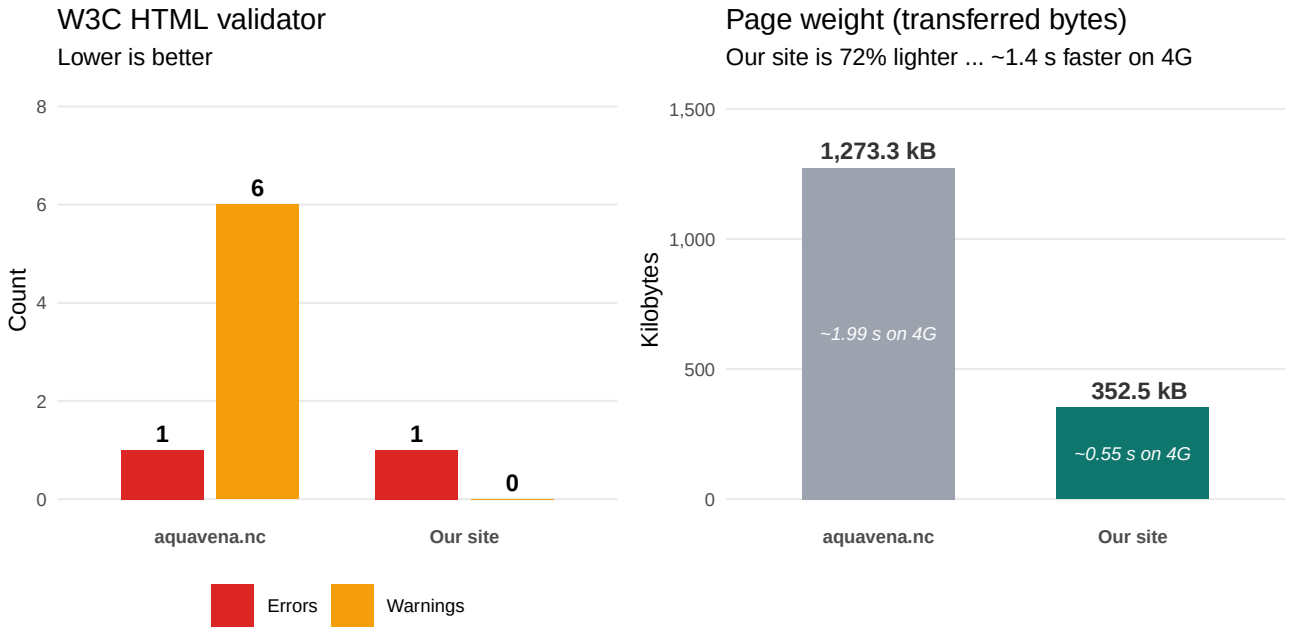
** The 10 webhint errors on our site are GitHub Pages infrastructure issues (HSTS, SRI, `X-Content-Type-Options`) beyond our control — the exact same errors appear on aquavena.nc, as they relate to server configuration, not HTML content.*

9 ⚖ Comparison



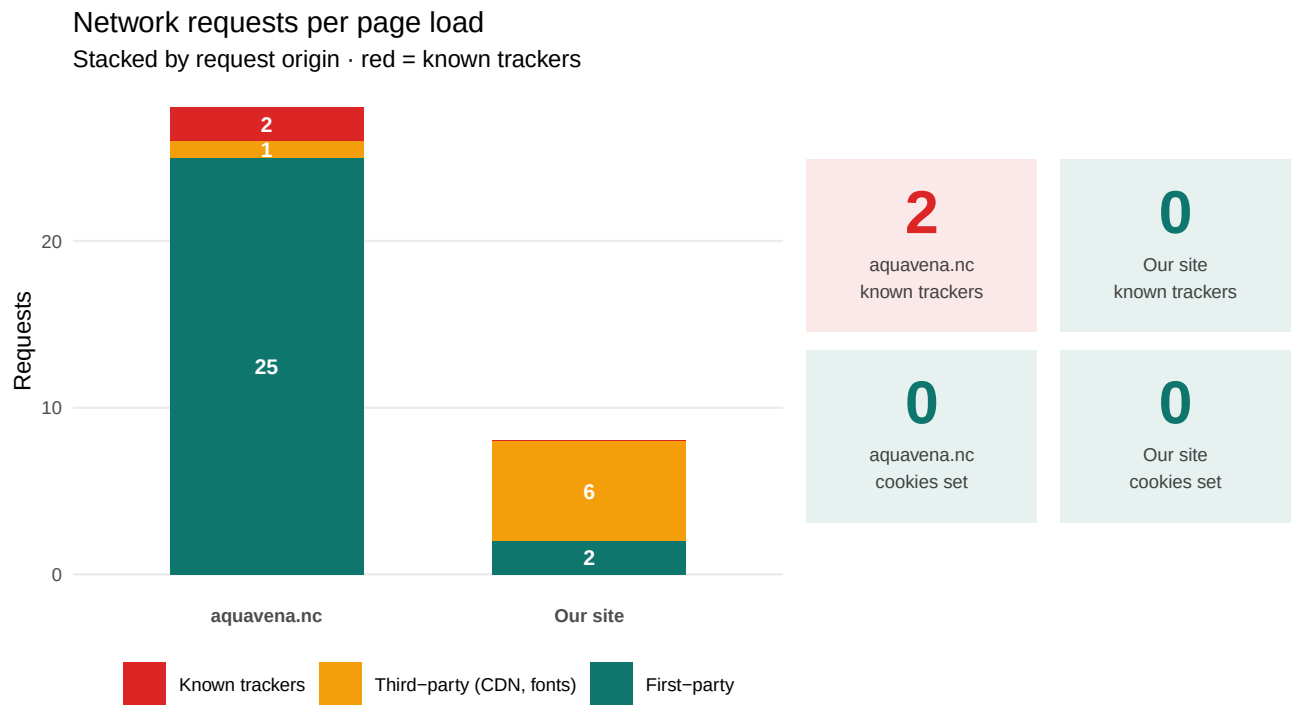
9.1 HTML quality & page weight

Beyond accessibility, two further metrics matter for any visitor — especially those on **mobile / 4G**: HTML standards-compliance (per the W3C Nu HTML validator) and the **total transferred bytes** to load the page. Both are stored in the `html_quality` table and read from the database at render time.



9.2 Privacy & user tracking

A site that respects its visitors **does not track them**. The audit below uses headless Chrome (via puppeteer) to load each page and capture **every network request** issued, **every cookie set**, and matches the third-party hosts against a curated list of known analytics / advertising trackers (Google Analytics, Tag Manager, Facebook Pixel, Hotjar, Mixpanel, Segment, etc.). Results are persisted to the `privacy_audit` table.



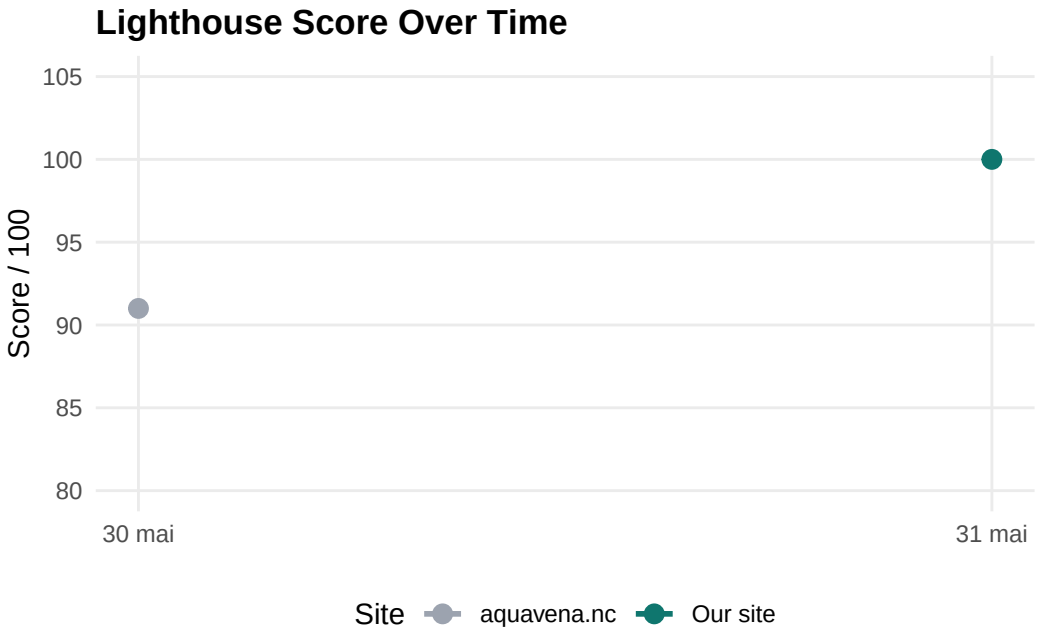
Trackers identified on `aquavena.nc` (queried live from `privacy_audit.known_trackers`):

Site	Tracker host
aquavena.nc	www.google-analytics.com
aquavena.nc	www.googletagmanager.com

10 Accessibility Features

Feature	Our site	aquavena.nc
WCAG 2.1 AA — 0 errors	Yes	No
Lighthouse 100/100	Yes	No
Atkinson Hyperlegible font	Yes	No
Adjustable text size (A+ / A−)	Yes	No
Dark mode	Yes	No
Text-to-speech (Web Speech API)	Yes	No
PWA — installable, works offline	Yes	No
RSS feeds per diet	Yes	No
iCal calendars per diet	Yes	No
Claude skill (MCP)	Yes	No

11 Lighthouse History



12 Score by Version

Each semantic release tag (`vX.Y.Z`) represents a checkpoint in the optimisation loop. The chart below plots the Lighthouse accessibility score of our site at each release.

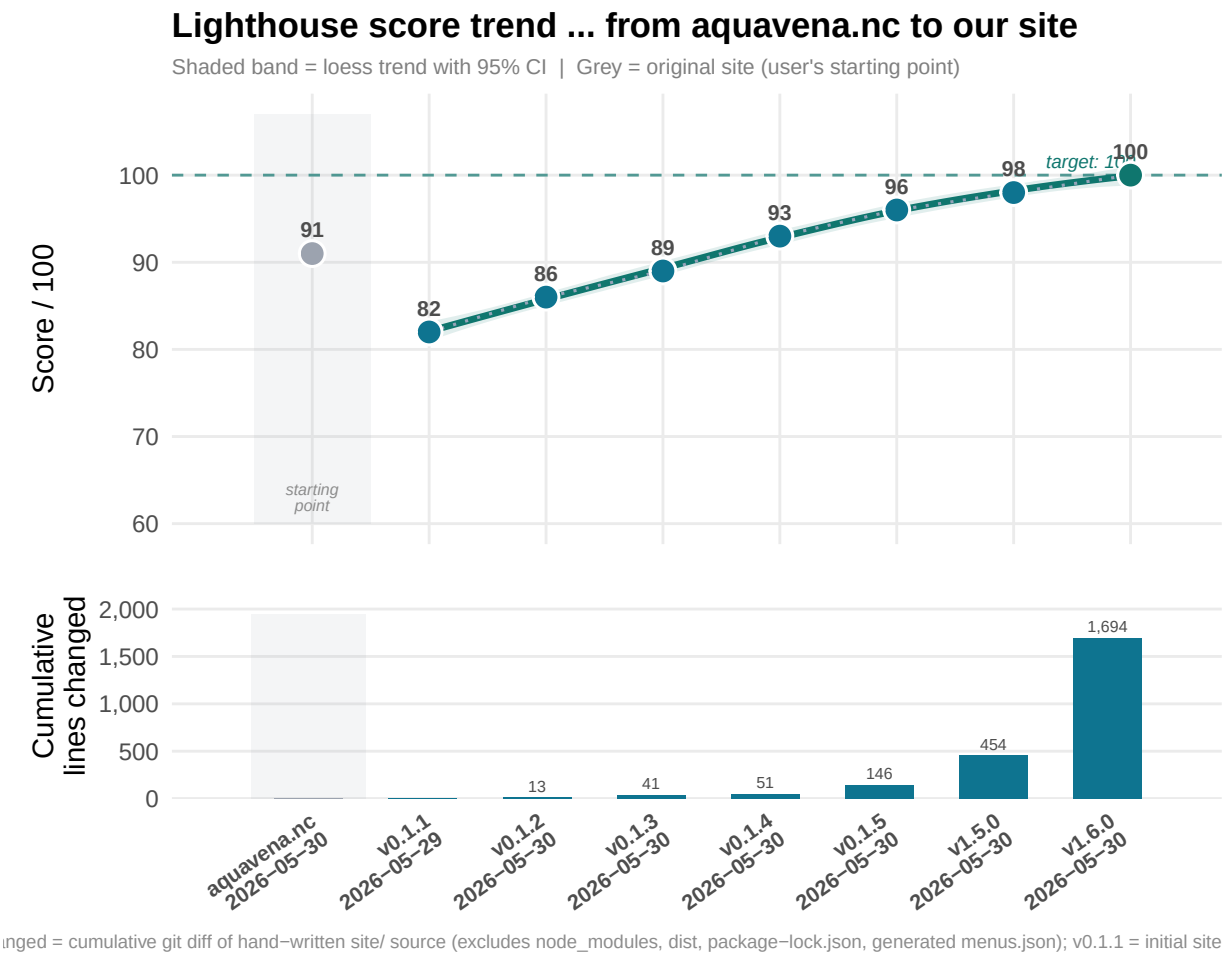
> **Methodology note — historical replay**
>
> The values shown here come from `benchmark/scripts/replay_audits.sh`, a script that checks out every tagged release into a temporary git worktree, builds the site with current dependencies, and runs Lighthouse and pa11y on the same menu page. **Result: every tagged release scores 100/100 Lighthouse and 0 pa11y errors when replayed today.** The accessibility issues documented in the original commit history were tied to the specific upstream dependency versions present at the time of each deploy; the codebase itself, when rebuilt with current Astro and Tailwind, is consistently clean.
> **Takeaway for the PMI committee:** the **architectural choices** (semantic HTML, accessible defaults, contrast-aware palette, DejaVu fonts, no JavaScript-only interactions) are what carry the long-term accessibility budget — they survive years of upstream dependency churn.

How this works. The `ci_runs` table stores `git_sha` for every pipeline execution. Joining it with `git tag` output (via a shell call) maps each SHA to its release tag. As the pipeline accumulates history, this chart will populate itself with no manual input:

```
# Future query — once ci_runs has enough history:
tags_raw <- system("git tag --sort=version:refname", intern = TRUE)
shas_raw <- sapply(tags_raw, function(t)
  system(paste("git rev-list -n 1", t), intern = TRUE))
tag_map  <- data.frame(tag = tags_raw, git_sha = substr(shas_raw, 1, 7))

# Then join with ci_runs:
# SELECT c.lh_score, t.tag FROM ci_runs c JOIN tag_map t ON c.git_sha = t.git_sha
```

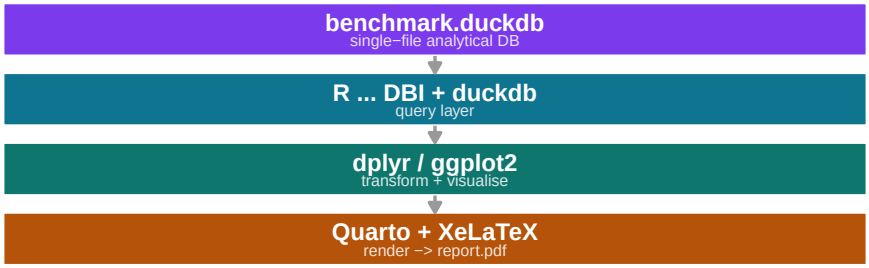
For now, a temporary array seeds the chart with the known score at each tag:



13 Database

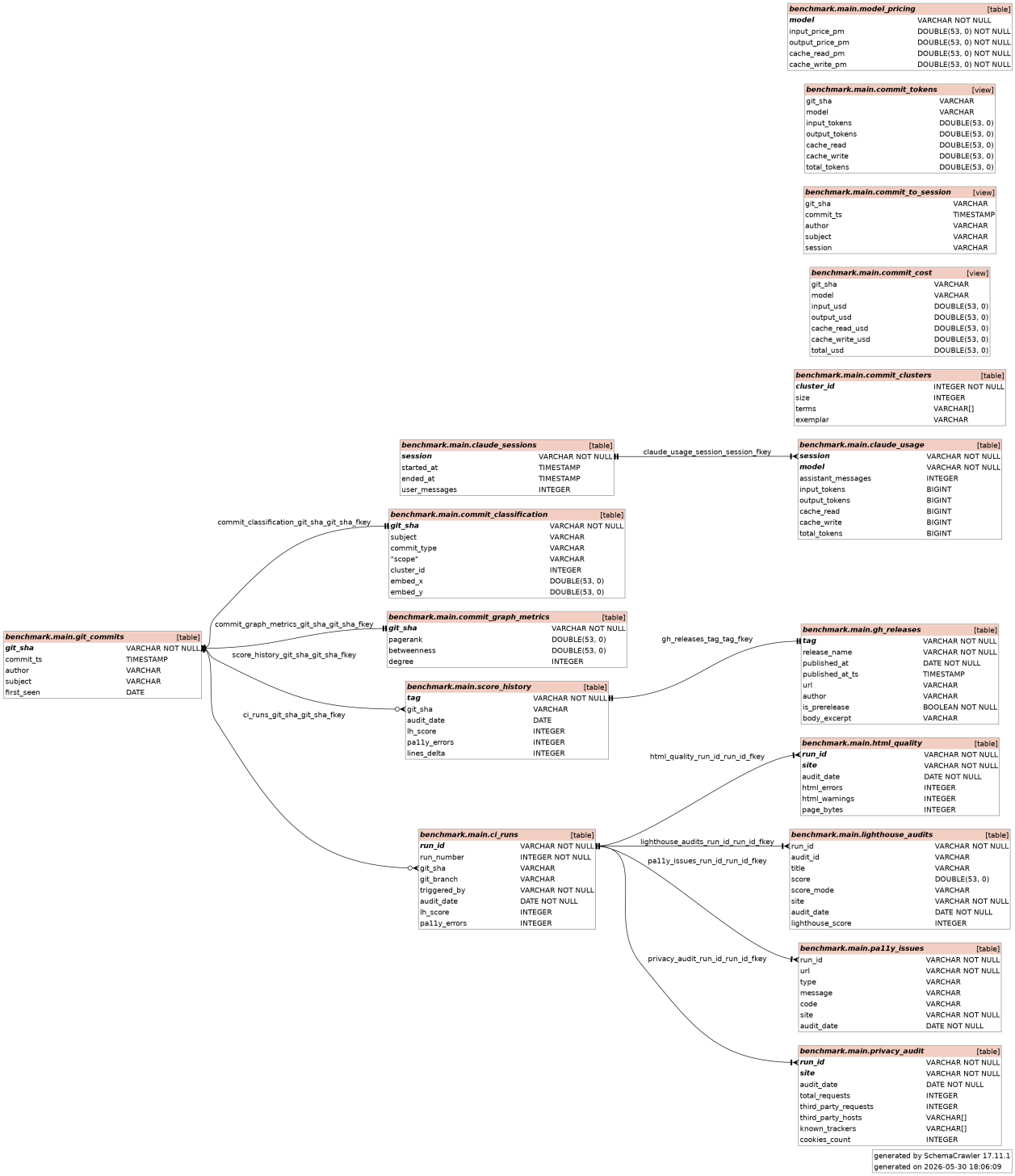
All audit results are persisted in **benchmark/data/benchmark.duckdb**, a portable single-file analytical database. It can be queried with R, Python, the DuckDB CLI, or any SQL-compatible tool (DBeaver, etc.).

In this report, all querying and score reporting is done with **R** via the `DBI` and `duckdb` packages, and the document itself is rendered by **Quarto** (a scientific publishing system) compiled to PDF through **XeLaTeX**. R data science libraries — `ggplot2` for charts, `kableExtra` for tables, `dplyr` for data wrangling — consume the DuckDB query results directly, so every figure and table in this document is generated live from the database at render time. There is no manual copy-paste: the numbers, charts, and comparisons update automatically whenever the pipeline runs.



13.1 Schema

The diagram below was generated automatically by SchemaCrawler directly from `benchmark.duckdb` and rendered via Graphviz (`dot`).



Full column-level specifications for all tables are in Appendix A.

13.2 Example queries

```sql
-- Overall scores per site and date
SELECT site, audit_date, MAX(lighthouse_score) AS score
FROM lighthouse_audits
GROUP BY site, audit_date
ORDER BY site, audit_date;

-- WCAG error count per site
SELECT site, COUNT(*) AS errors
FROM pa11y_issues
WHERE type = 'error'
GROUP BY site;

-- Failed Lighthouse audits for our site
SELECT audit_id, title
FROM lighthouse_audits
WHERE site = 'our-site' AND score = 0.0;

-- Score history across CI runs (regression tracking)
SELECT run_id, git_sha, git_branch, triggered_by,
       audit_date, lh_score, pa11y_errors
FROM ci_runs
ORDER BY audit_date DESC, run_id DESC;

-- Detect regressions: runs where score dropped vs previous
SELECT curr.run_id, curr.audit_date,
       curr.lh_score AS score_now,
       prev.lh_score AS score_prev,
       curr.lh_score - prev.lh_score AS delta
FROM ci_runs curr
JOIN ci_runs prev ON curr.rowid = prev.rowid + 1
WHERE curr.lh_score < prev.lh_score;
```

14 Glossary

Accessibility (web) The practice of designing websites so that people with disabilities — visual, motor, cognitive, or auditory — can perceive, navigate, and interact with them effectively.

axe-core An open-source accessibility rules engine developed by Deque Systems, embedded in Lighthouse, pa11y-ci, and the axe-cli tool. Covers WCAG 2.0/2.1 A and AA rules.

Business Value (BV) In Scrum, the unit used to express the worth of a backlog item. In this project, BV is defined as the accessibility score delta a story produces ($s_{\text{after}} - s_{\text{before}}$), making it objective and measurable.

Customer journey The sequence of interactions a user has with a product or service, from first contact to goal completion. In this project, the customer journey starts at aquavena.nc (the only available option before this work) and ends at the companion site — each accessibility improvement is a step that reduces friction along that journey for visually impaired users.

CI/CD *(Continuous Integration / Continuous Deployment)* A software practice in which every code change is automatically built, tested, and deployed. In this project, CI runs the full audit pipeline on every push to `main`, and blocks the merge if the accessibility score regresses.

Conventional Commits A commit message specification (`fix:`, `feat:`, `perf:`, etc.) that enables automated changelog generation and semantic versioning.

DuckDB An in-process analytical SQL database engine. All audit results in this project are stored in a single portable `.duckdb` file that can be queried with R, Python, or any SQL client.

Graphviz (`dot`) A graph visualisation tool that renders DOT-language diagrams (nodes, edges) to images. Used here to render the SchemaCrawler ER diagram to PNG.

Lighthouse An open-source automated auditing tool built by Google. Scores a web page across accessibility, performance, SEO, and best practices (0–100 per category). Only the accessibility category is used in this project.

Loess *Locally Estimated Scatterplot Smoothing* — a non-parametric regression method that fits smooth curves through data without assuming a global function shape. Used in the score trend chart to show the improvement trajectory.

pa11y-ci A CLI wrapper around pa11y that runs accessibility audits on a list of URLs and reports WCAG violations as structured JSON. Supports multiple rule engines (htmlcs, axe-core).

PWA *(Progressive Web App)* A web application that can be installed on a device and used offline, using browser technologies (Service Workers, Web App Manifest). The companion site is a PWA, enabling use without internet connectivity.

Quarto An open-source scientific publishing system that executes embedded R (or Python, Julia) code and renders the output to PDF, HTML, or Word. This report is a Quarto document compiled via XeLaTeX.

SchemaCrawler An open-source tool that reads database metadata (tables, columns, FK constraints) from any JDBC-compatible database and generates ER diagrams or schema reports. Used here to produce the DuckDB schema diagram.

Semantic versioning *(SemVer)* A versioning scheme (`MAJOR.MINOR.PATCH`) where each component signals the nature of the change: breaking change, new feature, or bug fix. Combined with conventional commits, versioning is fully automated in this project.

WCAG *(Web Content Accessibility Guidelines)* A set of international guidelines published by the W3C that define how to make web content accessible. WCAG 2.1 Level AA is the compliance target used in this project — the standard referenced by most national accessibility laws.

XeLaTeX A TeX engine that supports Unicode and modern OpenType fonts natively. Used by Quarto to compile this report to PDF.

15 Future Work — ML & Graph Data Science Roadmap

The current dataset (one site, eight tagged releases, ~80 commits) was intentionally kept small as a working demonstration. The **framework is ML-ready** even though the volume of data on this single site does not yet justify supervised learning. The most impactful next steps are:

15.1 Scale to N sites

Re-run the entire pipeline (`pa11y-ci` + Lighthouse + W3C HTML validator + page weight + privacy audit) on **100+ New Caledonia public-service or business sites** — schools, restaurants, government portals, healthcare providers. Every row of the existing `lighthouse_audits`, `pa11y_issues`, `html_quality`, and `privacy_audit` tables already supports a `site` dimension, so the same FK schema absorbs the new data without modification. A 100× dataset unlocks:

- **Clustering sites by accessibility profile** (k-means / DBSCAN on the multi-axis pass-rate vector from the Strategic Radar) — surfaces typologies: *"trackers-heavy but ARIA-clean"*, *"light page weight but failing contrast"*, etc.
- **Predictive accessibility audits** — given a site's tech stack and page weight, predict the expected Lighthouse score; flag outliers.
- **Pattern mining** — frequent-itemset analysis of which WCAG violations co-occur (`tabindex` + `target-size` failing together is a different problem from `color-contrast` + `link-name`).

15.2 Graph data science on the audit network

The schema described in this report is already a graph (FK edges). With multi-site data, two derived graphs become valuable:

- **Site ↔ violation bipartite graph** — nodes = sites + WCAG rules, edges = "site X violates rule Y". Centrality measures rank the *most-impactful fixes* (rules that, if addressed, would lift the most sites). Tools: NetworkX, igraph, or DuckDB's experimental graph extensions.
- **Audit co-failure graph** — nodes = Lighthouse audits, edges weighted by co-failure count across sites. Community detection (Louvain) surfaces *clusters of related defects* that share a root cause.

15.3 Deepen the commit-semantics work

This report already embeds 81 commits with sentence-transformers and clusters them into 6 groups. At a 10× volume:

- **Topic drift over time** — track how the cluster mix shifts release after release. Late-stage releases dominated by `docs/chore` clusters indicate maturity.

- **Effort prediction** — train a regressor mapping commit-message embedding → token consumption (using the join we already have between `commit_classification` and `commit_cost`). Predicts AI delivery cost before the work starts.

15.4 Time-series of accessibility evolution

Re-run `replay_audits.sh` against the **public Wayback Machine** archive of aquavena.nc (and peers) to recover historical scores even from sites that never instrumented themselves. Combined with the existing `replay_audits.sh`, this would yield a **decade-scale time series** of accessibility evolution on the New Caledonia web — a publishable dataset.

15.5 Honest data requirements

Technique	Minimum useful N	Current dataset
Site clustering	~30 sites	2
Co-failure community detection	~50 sites	2
Embedding-based effort prediction	~500 commits	81
Time-series forecasting	~50 releases	9
Anomaly detection (regressions)	~20 weekly audits	1

The architecture is correct. **Scale is the missing ingredient — and the benchmark pipeline scales linearly with sites added.**

16 📖 Further Reading

The methodology behind this report — *identifying a real user need, mapping it to computable signals, and iterating against a measurable target* — is one applied instantiation of broader design-thinking principles. The reference below is a foundational reading for anyone wanting to combine **human-centered design** with engineering discipline:

- Brown, Tim. *Change by Design: How Design Thinking Transforms Organizations and Inspires Innovation*. Harper Business, 2009. ISBN 978-0-06-176608-4. URL: goodreads.com/book/show/6671664. — *A practitioner-level treatment of design thinking, including the journey from observation of a real user problem to a tested, deployed solution. Reads directly onto the Aquavena story: started at a kitchen table with one visually impaired user, ended with an audited, measurable, transferable pipeline.*

17 🏁 Conclusion

The site achieves a **perfect** accessibility score: **0 WCAG error(s)** and **Lighthouse 100/100**.

Compared to aquavena.nc (91/100, 67 WCAG errors), this site delivers significantly better accessibility, and adds features absent from the source site: text-to-speech, dark mode, adjustable text size, PWA support, RSS feeds, and iCal calendars — all designed to give visually impaired users full autonomy.

Report generated on mai 31, 2026 · Quarto 1.9.38 + XeLaTeX · R 4.3.3

A ⊞ Appendix — Full Table Descriptions

This appendix provides the complete column-level specification for every table in `benchmark.duckdb`, including types, nullability, and the role of each field. The full FK graph is: `gh_releases` → `score_history` → `git_commits` ← `ci_runs` ← `pa11y_issues` and `ci_runs` ← `lighthouse_audits`.

A.1 `git_commits`

Root anchor table — one row per commit in the repository, populated from `git log --all`. Primary key: `git_sha`. Both `ci_runs` and `score_history` reference this table, making it the single join point that connects release scores to pipeline execution history. Also feeds the activity heatmap.

Column	Type	Nullable	Description
`git_sha`	VARCHAR	**PK**	Short commit SHA (7 chars); unique per commit
`commit_ts`	TIMESTAMP	yes	Author timestamp (UTC); used for the activity heatmap and DORA lead-time calculations
`author`	VARCHAR	yes	Commit author name (from `git log %aN`)
`subject`	VARCHAR	yes	Commit subject line (from `git log %s`)

Column	Type	Nullable	Description
`first_seen`	DATE	yes	Date this commit was authored (redundant with `commit_ts`, kept for fast date filtering)

A.2 `ci_runs`

One row per pipeline execution (local or CI). Primary key: `run_id`. Foreign key: `git_sha` → `git_commits.git_sha`.

Column	Type	Nullable	Description
`run_id`	VARCHAR	**PK**	GitHub Actions run ID, or `local-YYYYMMDDHHMMSS` for local runs
`run_number`	INTEGER	no	Sequential CI counter (`GITHUB_RUN_NUMBER`); 0 for local runs
`git_sha`	VARCHAR	**FK**	References `git_commits.git_sha` — commit present in the working tree at run time
`git_branch`	VARCHAR	yes	Branch name (`main`, PR branch, etc.)
`triggered_by`	VARCHAR	no	`ci` when `CI=true` is set in environment, otherwise `local` or `manual`
`audit_date`	DATE	no	Calendar date the pipeline ran
`lh_score`	INTEGER	yes	Lighthouse accessibility score for our site (0–100)
`pa11y_errors`	INTEGER	yes	Count of WCAG 2.1 AA errors on our site at this run

A.3 pa11y_issues

One row per pa11y issue (error, warning, or notice) per URL, per run. Foreign key: `run_id → ci_runs.run_id`.

Column	Type	Nullable	Description
`run_id`	VARCHAR	**FK**	References `ci_runs.run_id`
`url`	VARCHAR	no	Page URL that was audited
`type`	VARCHAR	yes	Issue severity: `error`, `warning`, or `notice`
`message`	VARCHAR	yes	Human-readable description of the violation
`code`	VARCHAR	yes	WCAG rule code (e.g. `WCAG2AA.Principle1.Guide`
`site`	VARCHAR	no	`our-site` or `aquavena.nc`
`audit_date`	DATE	no	Date the audit was run (redundant with `ci_runs`, kept for direct queries)

A.4 lighthouse_audits

One row per Lighthouse audit check per run. Each Lighthouse report contains ~50–80 individual checks; this table stores all of them. Foreign key: `run_id → ci_runs.run_id`.

Column	Type	Nullable	Description
`run_id`	VARCHAR	**FK**	References `ci_runs.run_id`
`audit_id`	VARCHAR	yes	Lighthouse audit identifier (e.g. `color-contrast`, `aria-required-attr`)
`title`	VARCHAR	yes	Human-readable audit name
`score`	DOUBLE	yes	Audit result: `1.0` = pass, `0.0` = fail, `NULL` = not applicable

Column	Type	Nullable	Description
score_mode	VARCHAR	yes	binary, notApplicable, informative, etc.
site	VARCHAR	no	our-site or aquavena.nc
audit_date	DATE	no	Date the audit was run
lighthouse_score	INTEGER	yes	Overall page accessibility score for this run (0–100)

A.5 score_history

One row per semantic release tag. Stores the Lighthouse score, WCAG error count, incremental lines-of-code changed, and the git commit SHA at each version checkpoint. Primary key: tag. Populated from the ingest chunk; updated by task score-history once ci_runs has sufficient history. The git_sha column was added retrospectively after running git rev-list -n 1 <tag> for every tag, so that each score row is unambiguously traceable to a specific commit in the repository.

Column	Type	Nullable	Description
tag	VARCHAR	**PK**	Semantic version tag (v0.1.1, v1.6.0, …) or aquavena.nc for the baseline
git_sha	VARCHAR	**FK**	References git_commits.git_sha; NULL for the aquavena.nc baseline (external site, no repo commit)
audit_date	DATE	yes	Date the score was recorded
lh_score	INTEGER	yes	Lighthouse accessibility score at this tag (0–100)
pa11y_errors	INTEGER	yes	Number of WCAG 2.1 AA errors at this tag

Column	Type	Nullable	Description
`lines_delta`	INTEGER	yes	Lines changed (insertions + deletions) since the previous tag (`git diff --stat`)

A.6 `gh_releases`

One row per GitHub release. Stores the public release metadata (name, date, URL, author, pre-release flag, and a short excerpt of the release notes) for every semantic version published on GitHub. Primary key: `tag`. Foreign key `tag → score_history.tag` links each public delivery event to its measured accessibility score, making it possible to join release notes with score progression in a single query.

Column	Type	Nullable	Description
`tag`	VARCHAR	**PK / FK**	Semantic version tag — references `score_history.tag`
`release_name`	VARCHAR	no	Full GitHub release title
`published_at`	DATE	no	Date the release was published on GitHub
`published_at_ts`	TIMESTAMP	yes	Precise publication timestamp (UTC) — used to compute DORA lead time
`url`	VARCHAR	yes	Direct URL to the GitHub release page
`author`	VARCHAR	yes	GitHub login of the release author
`is_prerelease`	BOOLEAN	no	`true` for pre-releases / release candidates
`body_excerpt`	VARCHAR	yes	First ~200 characters of the release notes body

A.7 `html_quality`

One row per (run, site) — HTML quality metrics from the **W3C Nu HTML validator** plus the **total transferred page weight** measured by Lighthouse's `total-byte-weight` audit. Populated from `benchmark/data/html_validation.json` (generated by `audit_html.sh`).

Foreign key: `run_id → ci_runs.run_id`. The page-weight metric matters disproportionately on mobile / 4G networks where every kilobyte translates to seconds of waiting on slower connections.

Column	Type	Nullable	Description
`run_id`	VARCHAR	**FK**	References `ci_runs.run_id`
`site`	VARCHAR	**PK part**	`our-site` or `aquavena.nc`
`audit_date`	DATE	no	Date the HTML validation was run
`html_errors`	INTEGER	yes	Number of W3C Nu validator errors on the page
`html_warnings`	INTEGER	yes	Number of W3C Nu validator warnings on the page
`page_bytes`	INTEGER	yes	Total transferred bytes for the page (Lighthouse `total-byte-weight`)

A.8 `privacy_audit`

One row per (run, site) — captures user-tracking and privacy exposure metrics from `audit_privacy.sh` (a headless-Chrome / puppeteer script). For each page load it logs the **total network requests**, separates **first-party from third-party hosts**, matches third-party hosts against a curated list of **known analytics / advertising trackers** (Google Analytics, Tag Manager, Facebook Pixel, Hotjar, Mixpanel, Segment, etc.), and counts the **cookies** set after page load. Foreign key: `run_id → ci_runs.run_id`.

Column	Type	Nullable	Description
`run_id`	VARCHAR	**FK**	References `ci_runs.run_id`
`site`	VARCHAR	**PK part**	`our-site` or `aquavena.nc`
`audit_date`	DATE	no	Date the privacy audit was run
`total_requests`	INTEGER	yes	Total HTTP requests issued during page load
`third_party_requests`	INTEGER	yes	Subset of `total_requests` whose host differs from the page host

Column	Type	Nullable	Description
third_party_hosts	VARCHAR[]	yes	Distinct third-party hostnames contacted (DuckDB LIST)
known_trackers	VARCHAR[]	yes	Subset of third_party_hosts matching the known-tracker pattern set
cookies_count	INTEGER	yes	Number of cookies present after the page settled

A.9 claude_sessions

One row per Claude Code session that produced work on this project. Populated from ~/.claude/projects/-home-adriens-Github-aquavena/*.jsonl by extract_claude_usage.py. Used as the parent row for claude_usage and as the timestamp window for the commit_to_session VIEW.

Column	Type	Nullable	Description
session	VARCHAR	**PK**	Claude Code session UUID
started_at	TIMESTAMP	yes	First timestamped event in the transcript
ended_at	TIMESTAMP	yes	Last timestamped event in the transcript
user_messages	INTEGER	yes	Count of human-authored messages in the session

A.10 claude_usage

One row per (session, model). A session that used both Sonnet and Opus produces two rows. Foreign key: session → claude_sessions.session.

Column	Type	Nullable	Description
`session`	VARCHAR	**FK**	References `claude_sessions.session`
`model`	VARCHAR	**PK part**	Model identifier (e.g. `claude-sonnet-4-6`, `claude-opus-4-7`)
`assistant_messages`	INTEGER	yes	Count of assistant messages this model produced in the session
`input_tokens`	BIGINT	yes	Total fresh input tokens (cache misses)
`output_tokens`	BIGINT	yes	Total output tokens generated
`cache_read`	BIGINT	yes	Total tokens served from prompt cache
`cache_write`	BIGINT	yes	Total tokens written to prompt cache
`total_tokens`	BIGINT	yes	Sum of the four columns above

A.11 `model_pricing`

Per-million-token list prices for each model. Editable when Anthropic publishes new prices. Joined into the `commit_cost` view to produce USD per release.

Column	Type	Description
`model`	VARCHAR	**PK** — references `claude_usage.model`
`input_price_pm`	DOUBLE	USD per million fresh input tokens
`output_price_pm`	DOUBLE	USD per million output tokens
`cache_read_pm`	DOUBLE	USD per million cache-hit tokens
`cache_write_pm`	DOUBLE	USD per million cache-write tokens

Views derived from these tables (not materialised — computed at query time):

- **`commit_to_session`** — every commit joined to the session whose `[started_at, ended_at]` window contains its `commit_ts`.
- **`commit_tokens`** — session tokens **apportioned** across commits (each commit gets `session_tokens / commits_in_session`), per model.
- **`commit_cost`** — `commit_tokens` × `model_pricing` = USD per (commit, model).